Deep Learning for Computer Vision

Day 9 – Autoencoders, GAN, GNN

Lecturer: Diego Borro

Computer Science PhD

Vision and Robotics group at CEIT

dborro@ceit.es

Timetable

Day / Hour	Day 5	Day 6	Day 7	Day 8	Day 9	Day 10
09:00-10:00	Optimizers	ViT	Object detection	Object segmentation (FCN)	Autoencoders	3D networks fundamentals
10:00-11:00	Basics Datasets Metrics	XAI	Datasets Metrics	Datasets Metrics	GAN	3D points cloud
11:00-12:00	CNN Architectures for classification	RNN – LSTM	Architectures R-CNN	U-Net Mask R-CNN	GNN	3D classification
12:00-13:30	CNN Architectures Transfer Learning	Graphics cards	SSD YOLO	Other uses	Practical cases	Practical cases

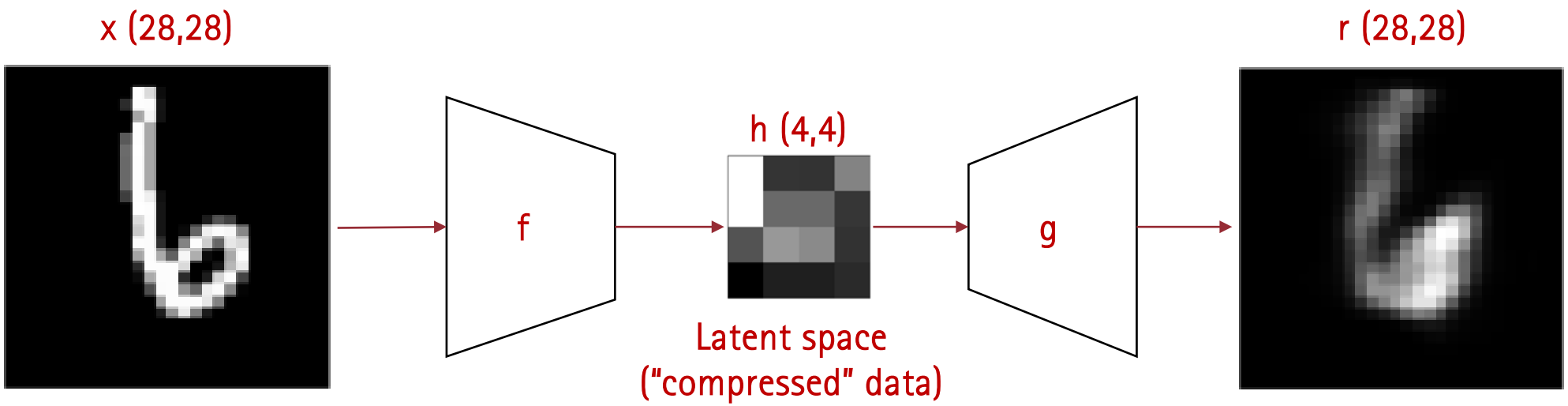
Index

1. Autoencoders
    1. Introduction to AutoEncoders
    2. Single layer AutoEncoders
    3. Deep AutoEncoders
    4. Convolutional autoencoders
2. Generative Adversarial Networks (GAN)
    1. Introduction to GANs
    2. Dense GAN
    3. Convolutional GAN
3. Graph Neural Networks (GNN)

1. Autoencoders

1.1 Introduction to AutoEncoders

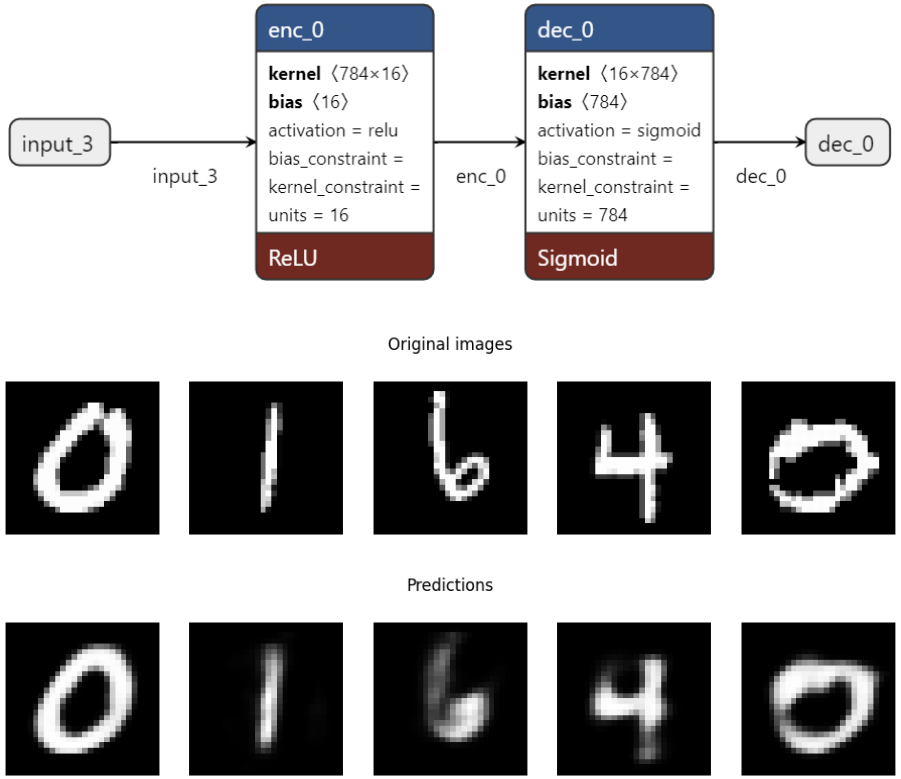
- AutoEncoders are Neural Networks **trained to attempt** to **copy** their **input** to their **output**.
- The network consist in two parts:
  - **Encoder block**: $h = f(x)$
  - **Decoder block**: $r = g(h)$
- The encoder function **generates a latent space "h"** which is a **lower resolution** representation of the input
- The decoder function **generates a recontruction "r"** from the latent representation **h**
- As the **dimensions** of the **original images are reduced** in the latent space, AutoEncoders are **forced to prioritize features** of the original image to represent it as a **new one**



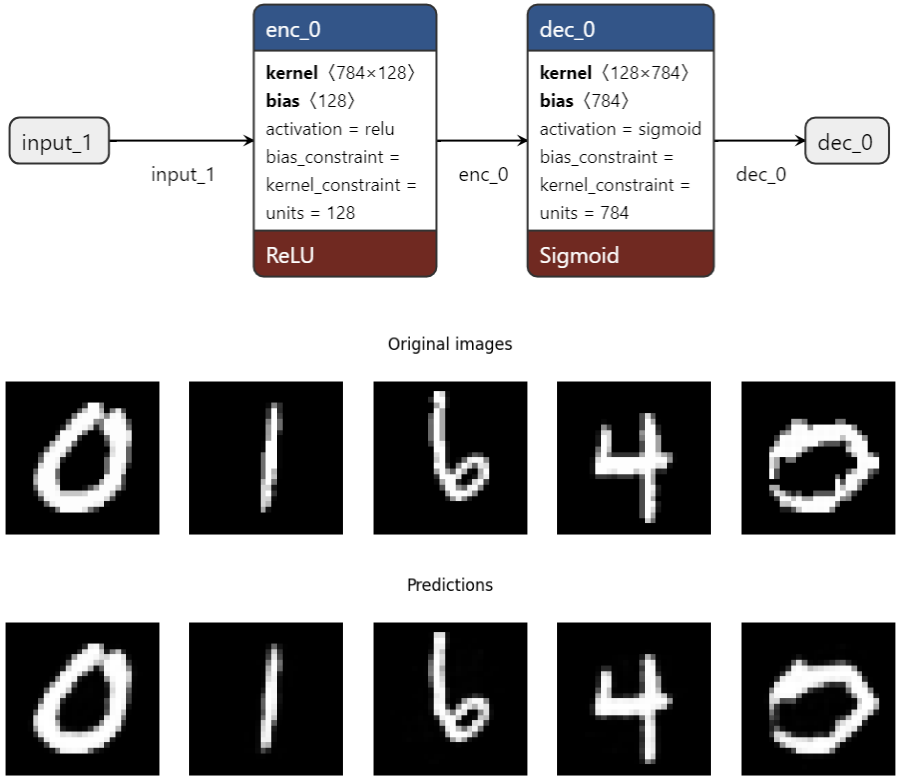
1.2 Simple layer AutoEncoders

- If the dimensions of the **latent space is bigger**, the reconstruction **should** be better.
- **Generally**, the dimensions of the input and the reconstruction are the same.
- The **input** and the **ground truth** sets for training **are the same**, as the **desired output is the input**.

Neurons: 16 → 784

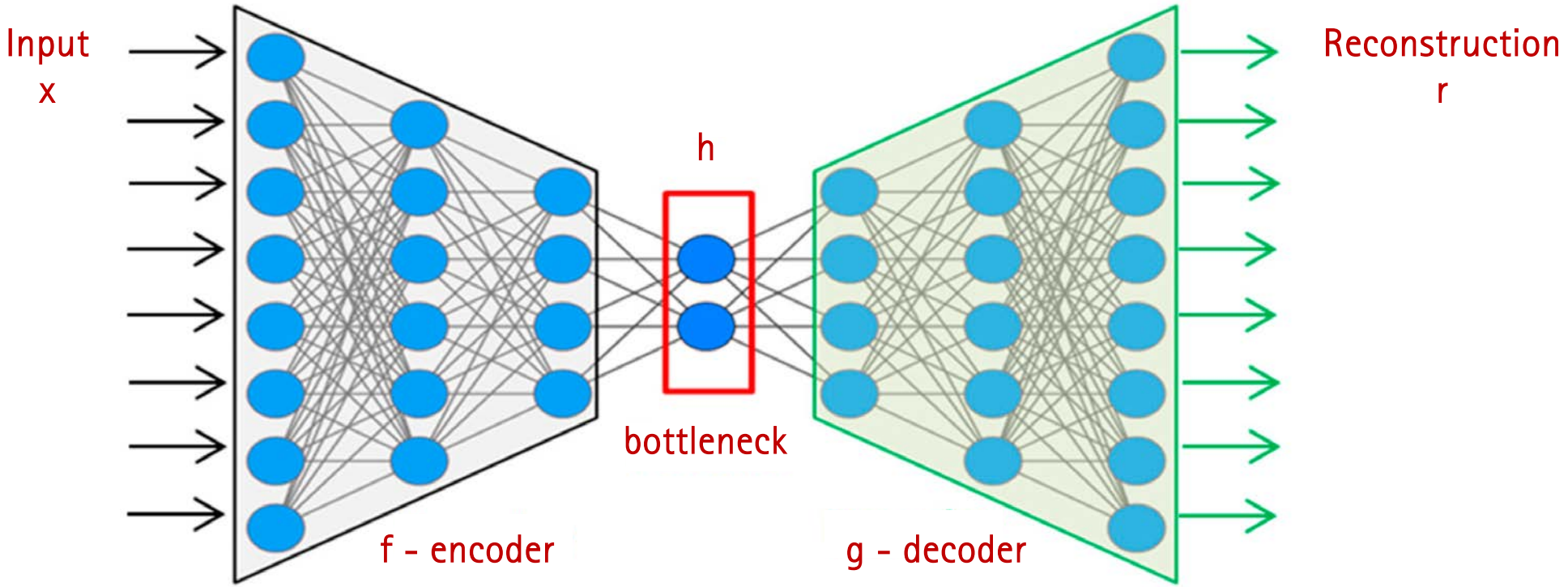


Neurons: 128 → 784



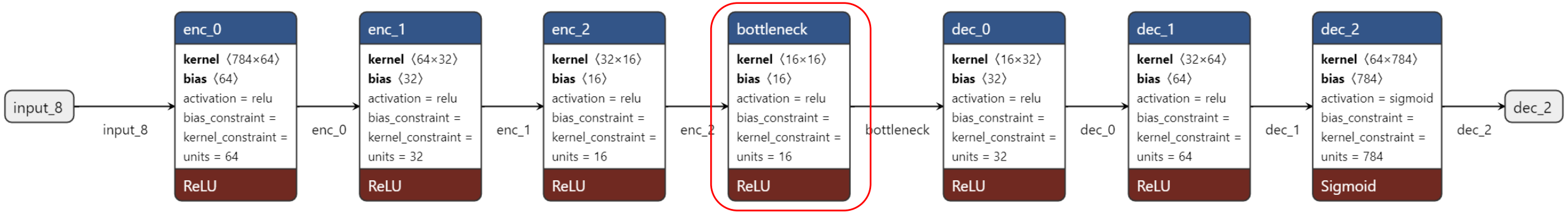
1.3 Deep AutoEncoders

- Deep AutoEncoders use **more than one** layer in **both** the **encoder** and the **decoder** blocks
- Bottleneck contains the **lowest resolution** representation of the input **(h – latent space)**, therefore **it must capture the most important features** of the input



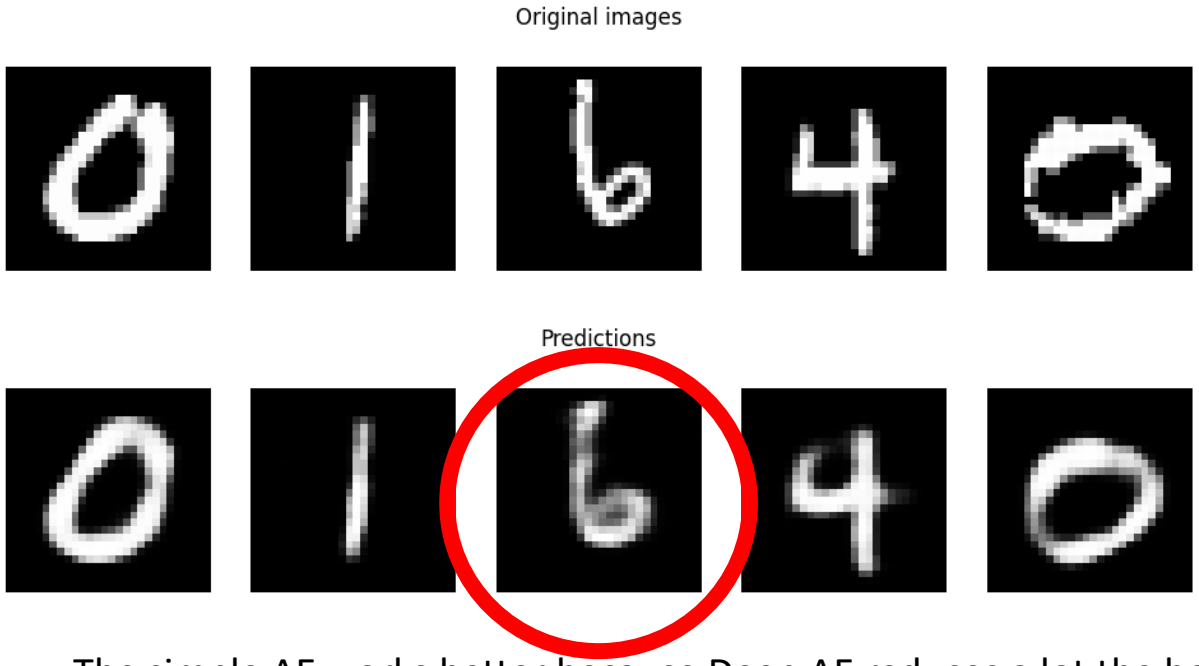
1.3 Deep AutoEncoders

Neurons: 784 → 64 → 32 → 16 → 16 → 32 → 64 → 784

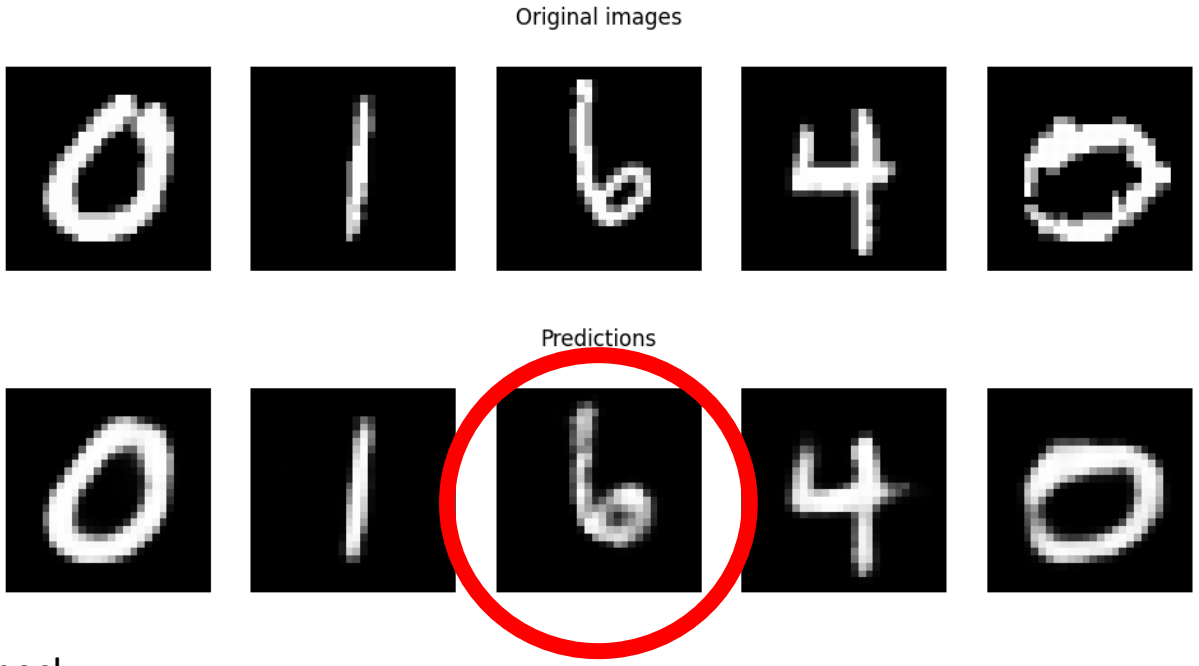


1.3 Deep AutoEncoders

Neurons: 784 → 64 → 32 → 16 → 16 → 32 → 64 → 784



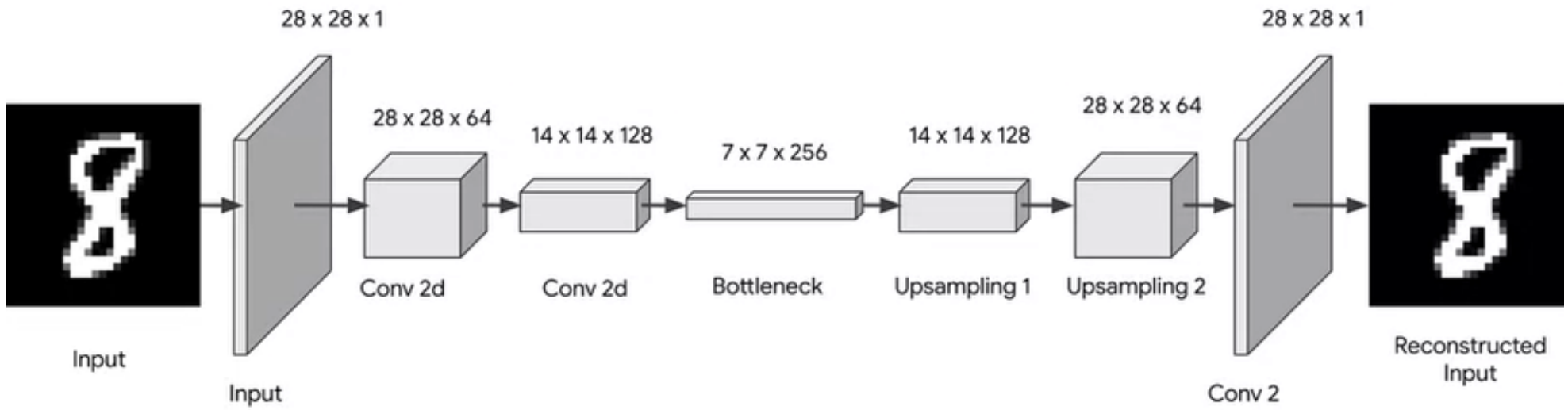
Neurons: 784 → 128 → 64 → 32 → 32 → 64 → 128 → 784



- The simple AE works better because Deep AE reduces a lot the bottleneck...
- Deep AE is not ideal for this task, due to the limitations of dense networks for working with images...

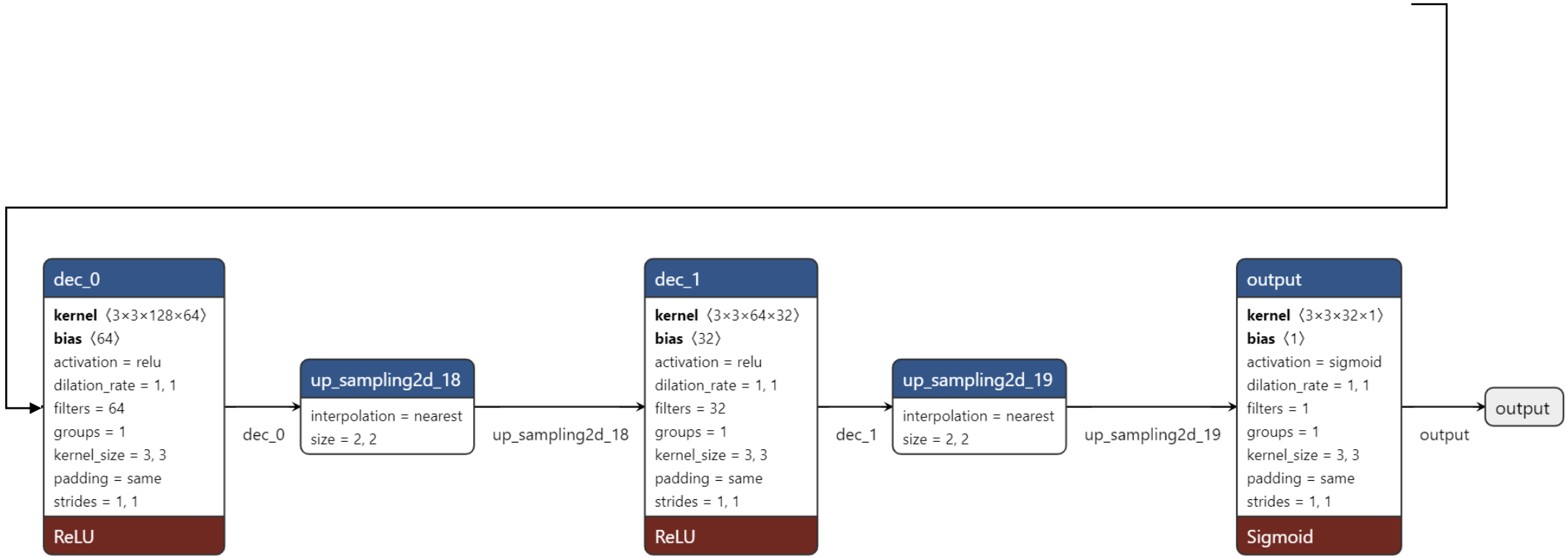
1.4 Convolutional AutoEncoders

- In **Convolutional AutoEncoders** the **Dense** layers are changed to **Convolutional** layers
- **Dimension reduction** is achieved with **Pooling** layer and/or **stride>1**
- **Dimension increase** is achieved with **Upsample** and/or **Deconvolution** layer
- The **Output** layer contains only **1 filter for grayscale** images and **3 filters for RGB** ones



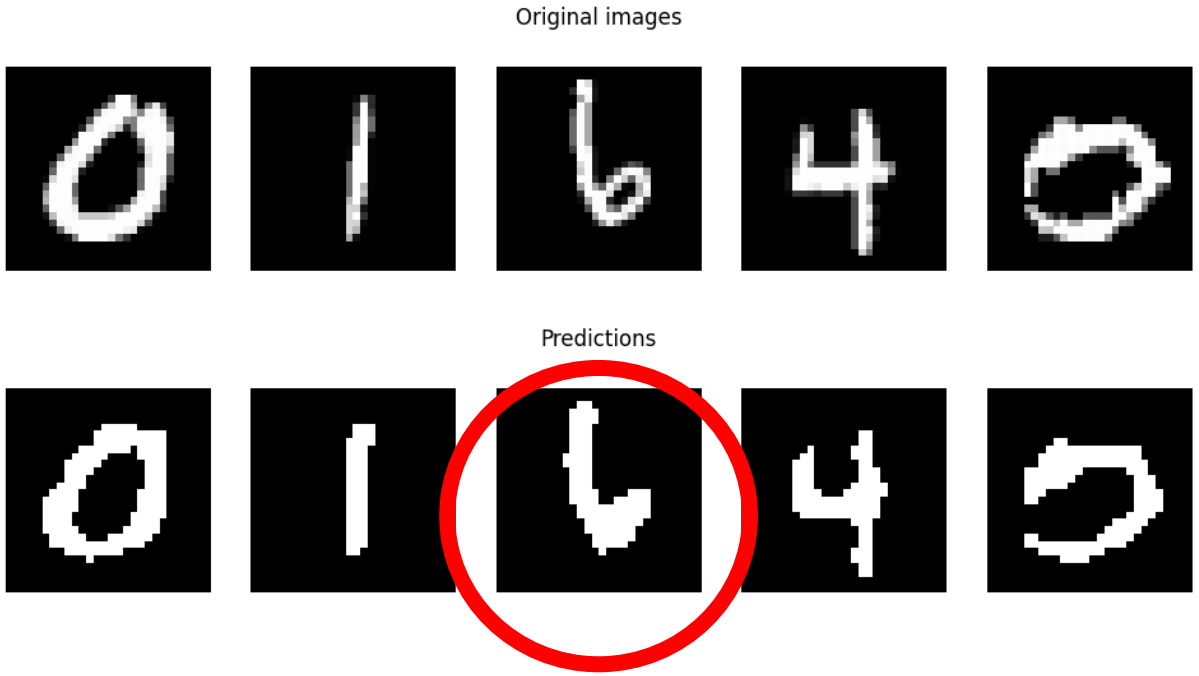
1.4 Convolutional AutoEncoders

FMs: 32 → 64 → 128 → 64 → 32 → 1

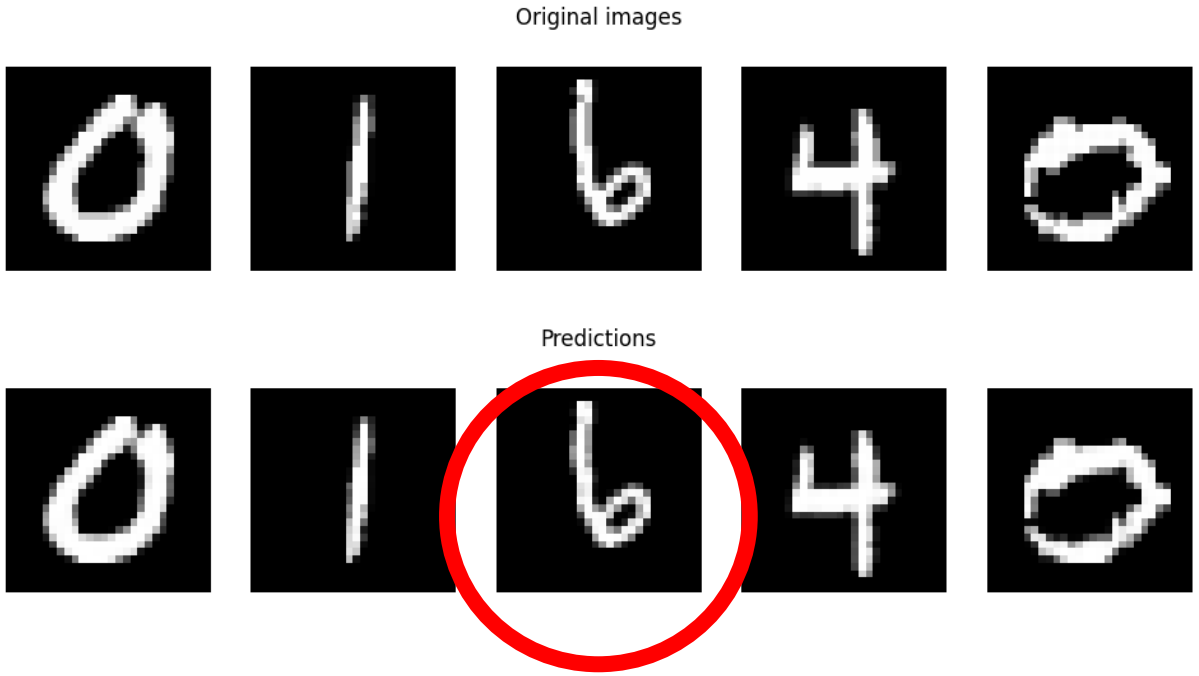


1.4 Convolutional AutoEncoders

FMs: 32 → 64 → 128 → 64 → 32 → 1



FMs: 64 → 128 → 256 → 128 → 64 → 1



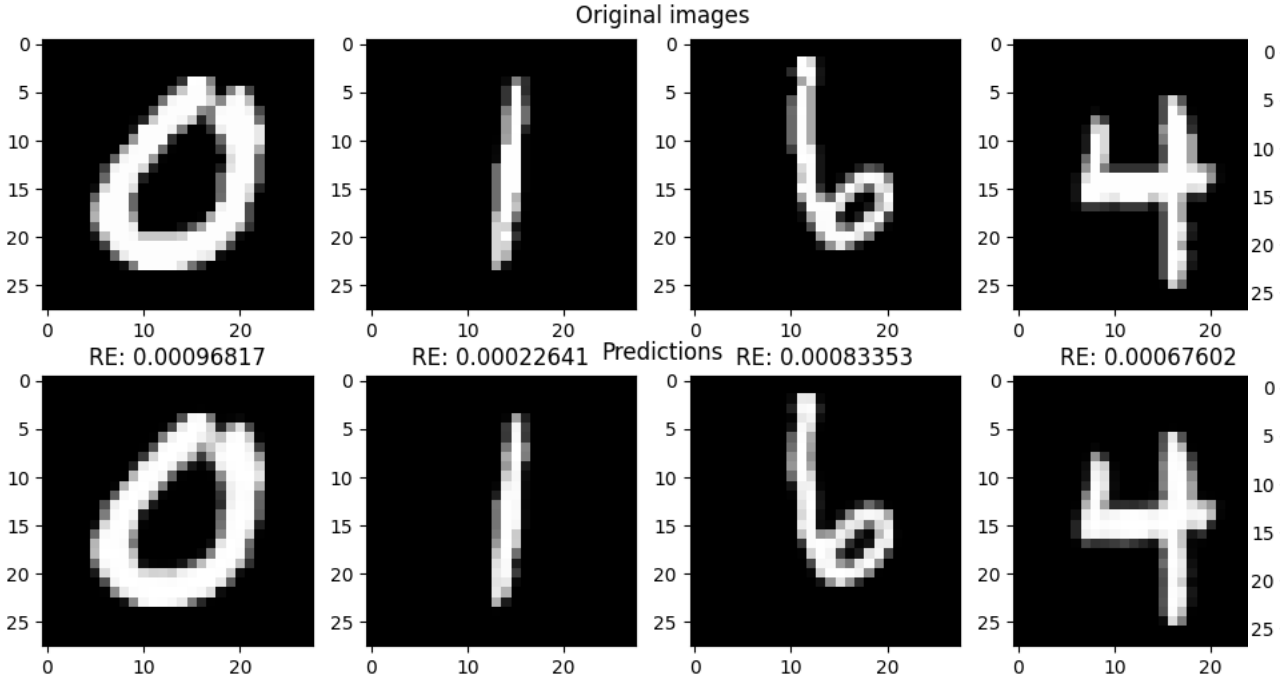
Reconstruction Error (RE)

The operations is based on the calculation of the Reconstruction Error, wich is calculated as shown in the formula:

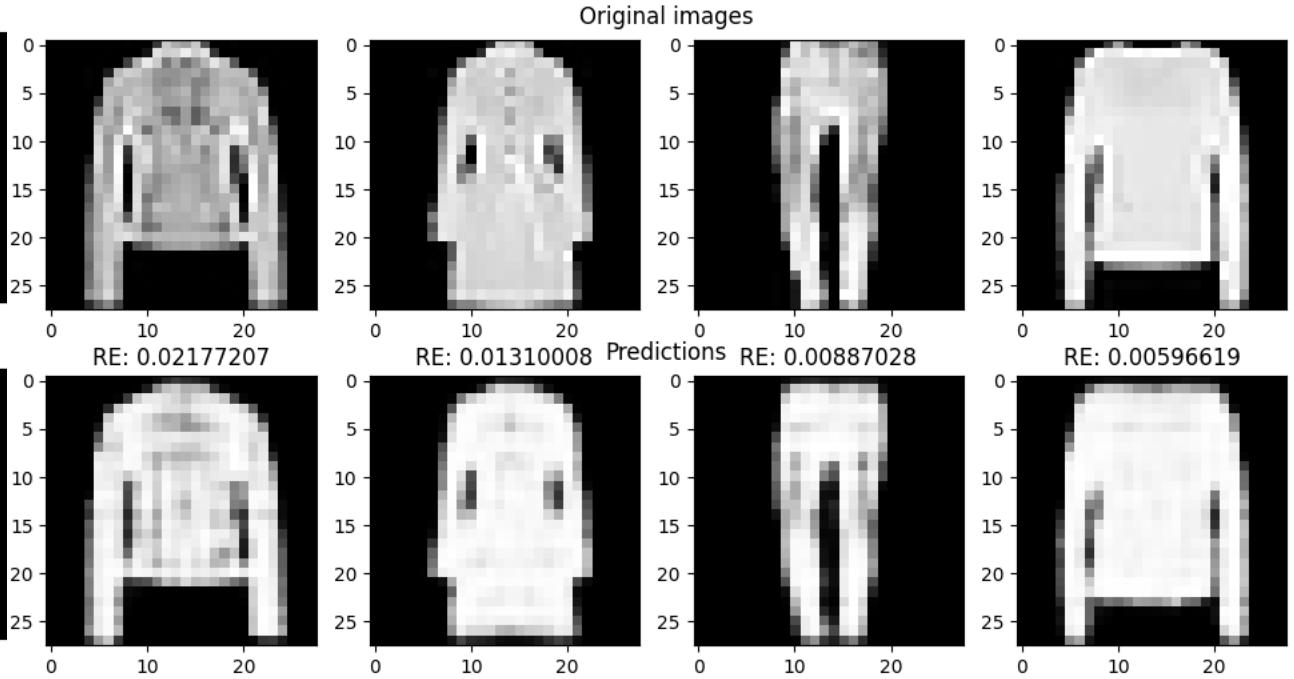
$$\mathrm{RE} = \sum_{i=1}^{w \cdot h} \frac{(x_i - x_{\mathrm{pred}})^2}{m}$$

The AutoEncoder trained in MNIST dataset, will have bigger Reconstruction Error in other images:

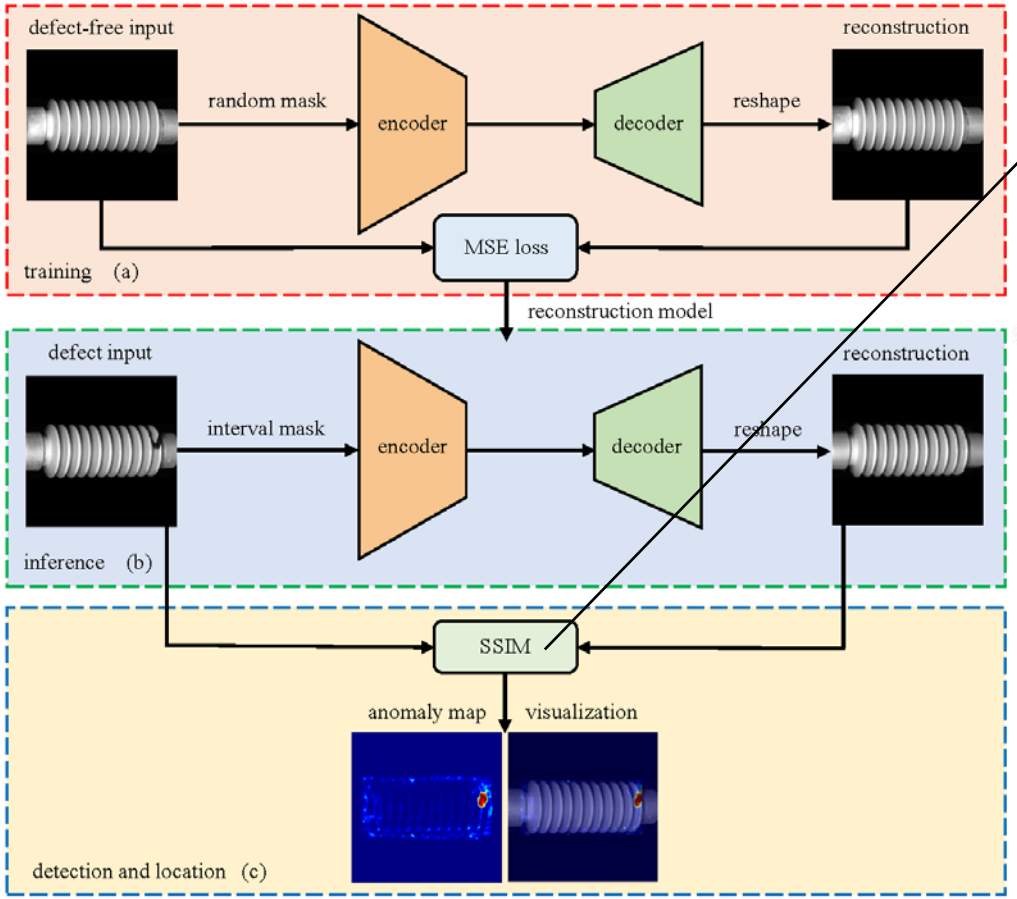
(RE ~ 6,8 * 10^{-4})



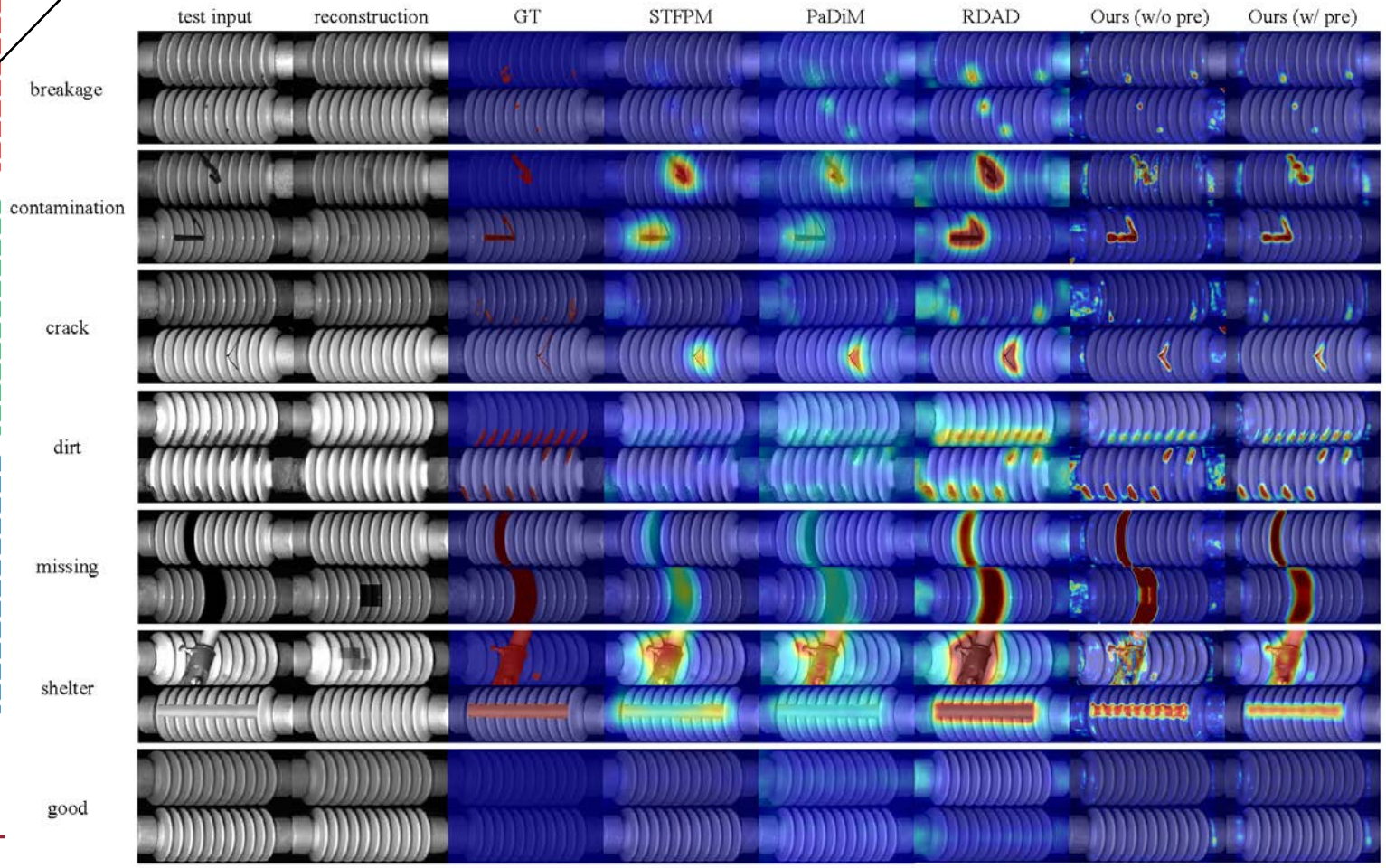
(RE ~ 1,2 * 10^{-2})



Example: Anomaly Detection (railway insulator defect detection)

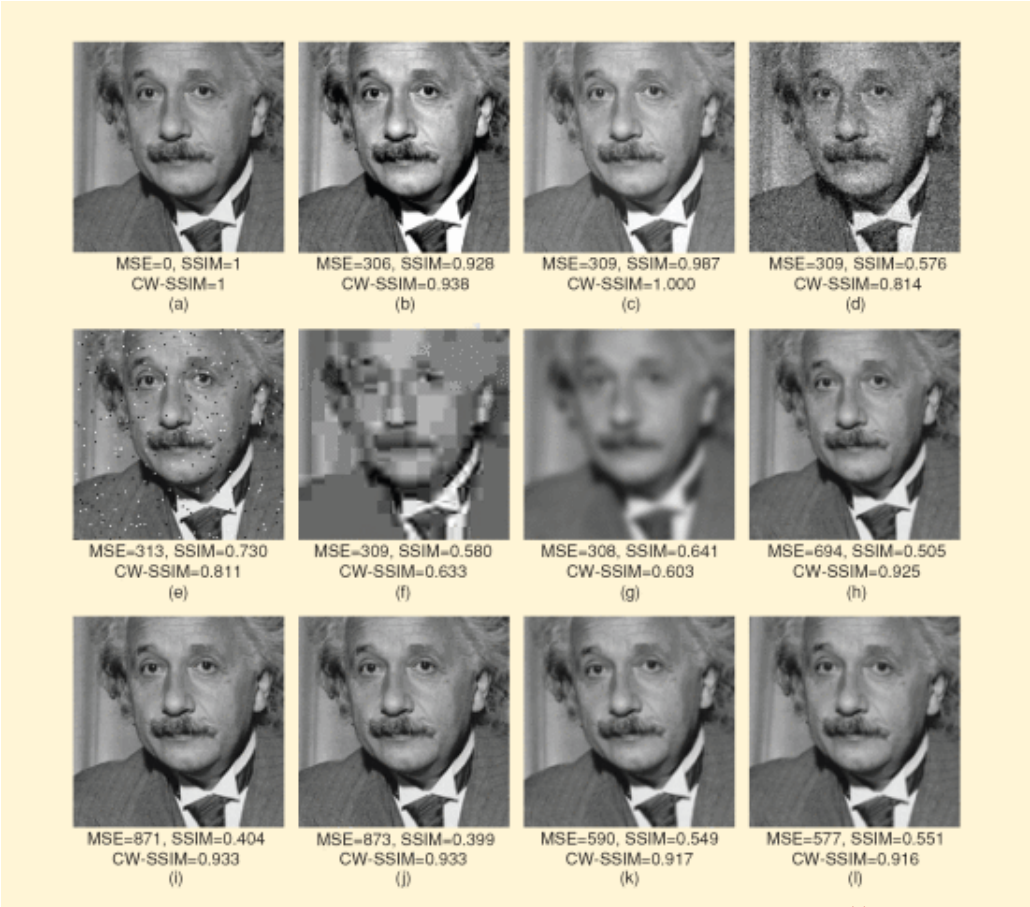


SSIM measures signal fidelity: it refers to the accuracy and quality with which a signal reproduces or represents the original information.



1.1 Introduction to AutoEncoders

- SSIM (Structural Similarity Index Measure)
  - Does the MSE really measure signal fidelity?



Comparison of image fidelity measures for "Einstein" image altered with different types of distortions. (a) Reference image. (b) Mean contrast stretch. (c) Luminance shift. (d) Gaussian noise contamination. (e) Impulsive noise contamination. (f) JPEG compression. (g) Blurring. (h) Spatial scaling (zooming out). (i) Spatial shift (to the right). (j) Spatial shift (to the left). (k) Rotation (counter-clockwise). (l) Rotation (clockwise).

It measures the similarities of three elements of the image patches

Window — Luminance comparison measure — Contrast — Structure

$$SSIM(x,y) = l(x,y)c(x,y)\,s(x,y)$$

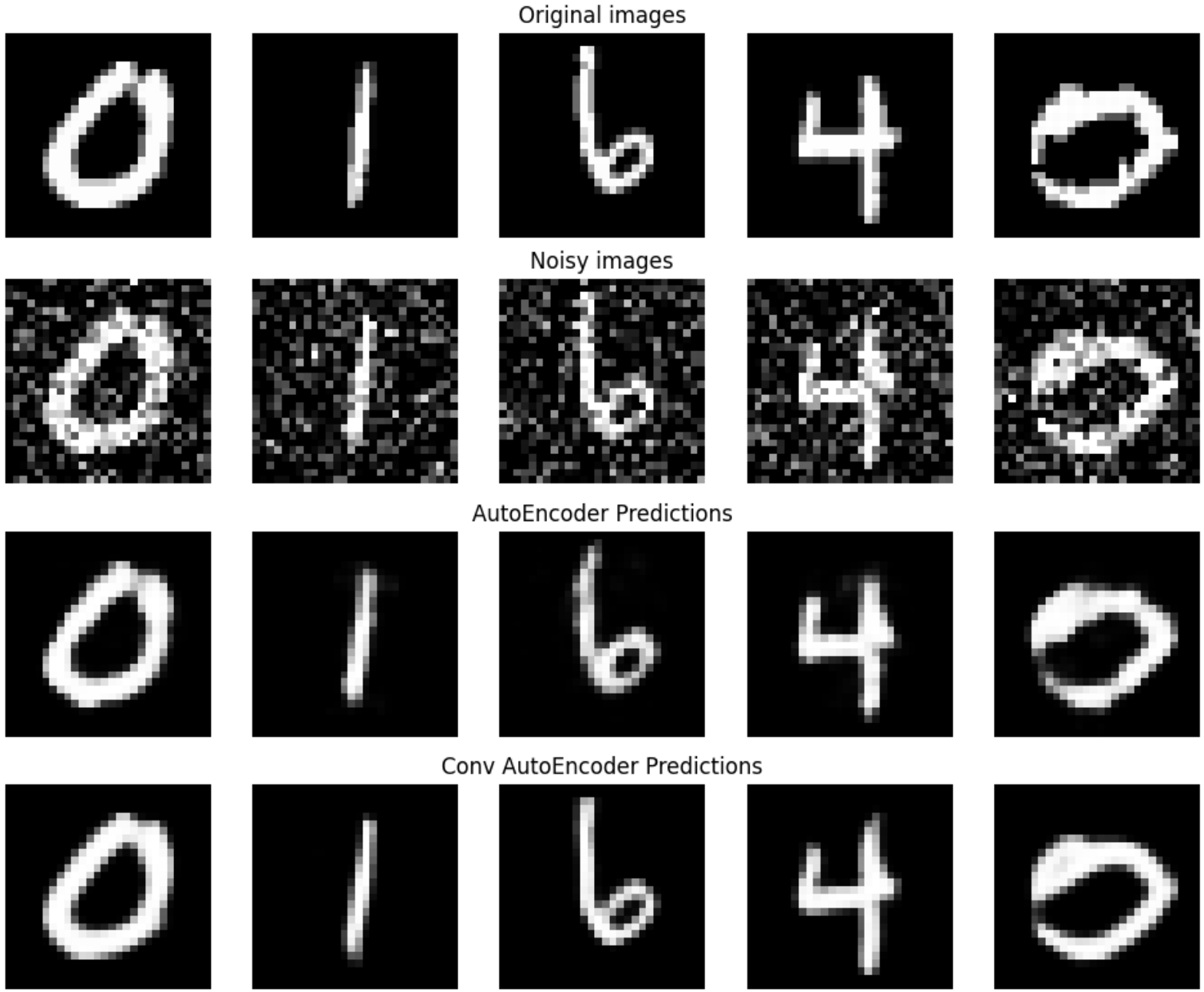
$$l(x,y) = \frac{2\mu_x\mu_y + c_1}{\mu_x^2 + \mu_y^2 + c_1}$$

$$c(x,y) = \frac{2\sigma_x\sigma_y + c_2}{\sigma_x^2 + \sigma_y^2 + c_2}$$

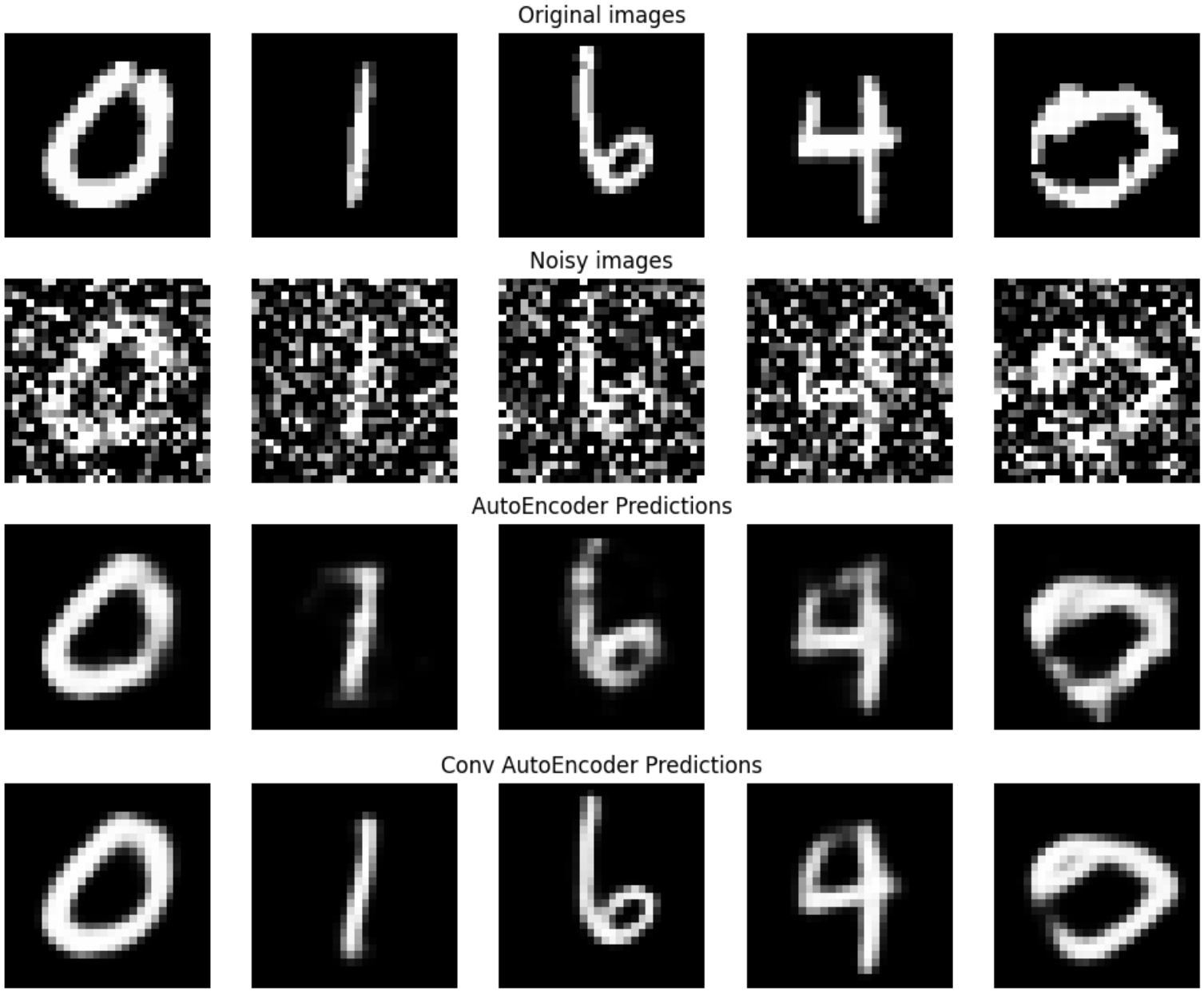
$$s(x,y) = \frac{\sigma_{xy} + c_3}{\sigma_x\sigma_y + c_3}$$

- μ_x: pixel sample mean of x
- μ_y: pixel sample mean of y
- σ_x: standard deviation of x
- σ_y: standard deviation of y
- σ_{xy}: covariance of x and y
- c_1: $(K_1L)^2$
- c_2: $(K_2L)^2$
- c_3: $c_2/2$

  Small positive stabilizing constants to avoid numerical instability
- L: dynamic range of the pixel-values
  - $(2^{\#bits_per_pixel} - 1)$
- K_1=0.01
- K_2=0.03



2. Practical case: MNIST denoising, low noise



3. Practical case: MNIST denoising, high noise

2. Generative Adversarial Networks (GAN)

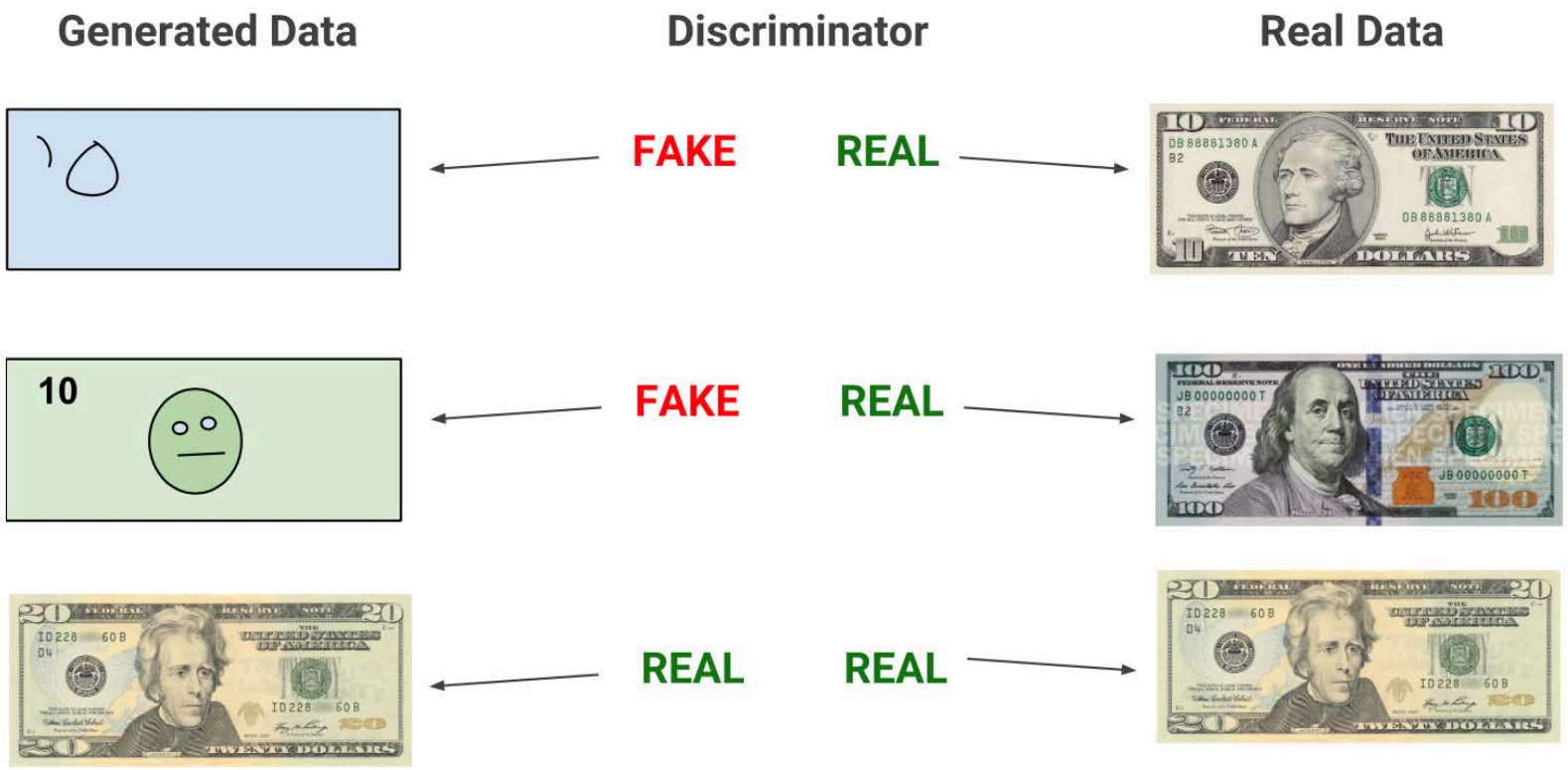
2. Introduction to GANs

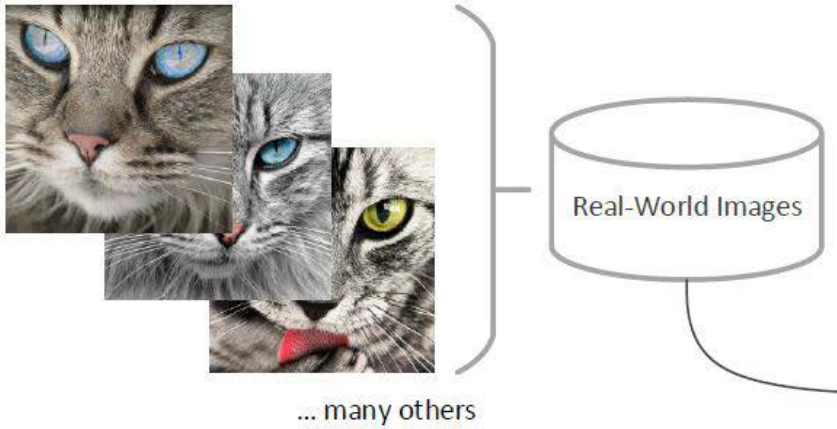
Generative adversarial networks (GANs) are an exciting recent innovation in machine learning. GANs are **generative models**: they create new data instances that resemble your training data. For example, GANs can create images that look like photographs of human faces, even though the faces don't belong to any real person

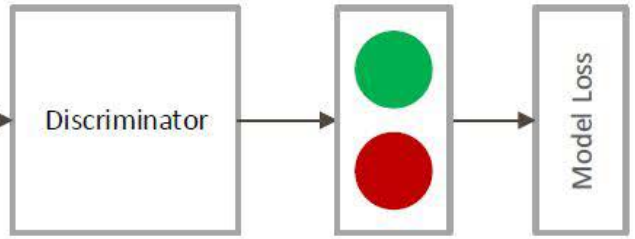
GANs achieve this level of realism by pairing a **generator**, which learns to produce the target output, with a **discriminator**, which learns to distinguish true data from the output of the generator. **The generator tries to fool the discriminator, and the discriminator tries to keep from being fooled**

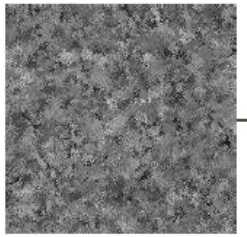
Generator: learns to generate plausible data. The generated instances become negative training examples for the discriminator

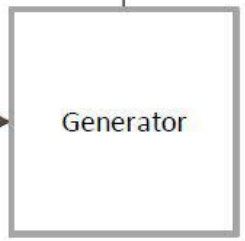
Discriminator: learns to distinguish the generator's fake data from real data. The discriminator penalizes the generator for producing implausible results





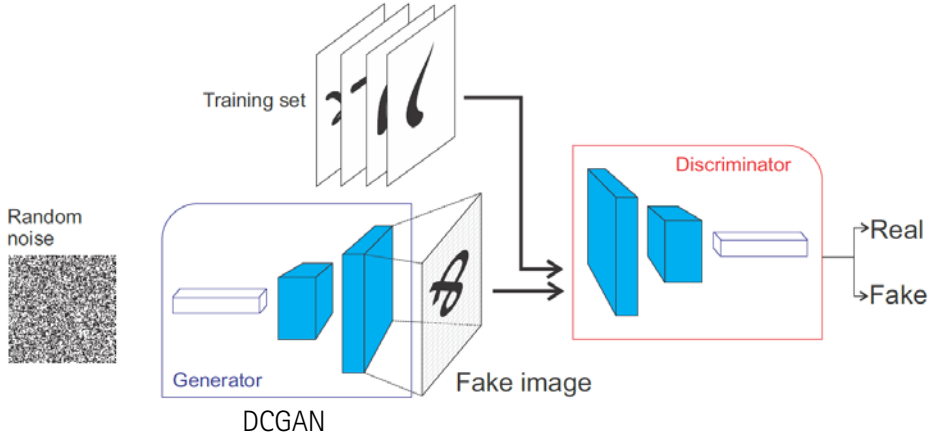


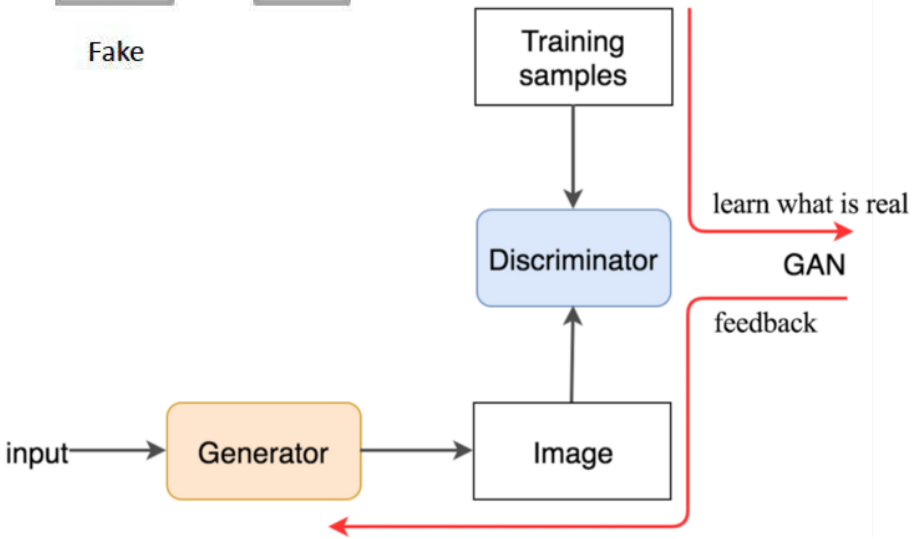




Latent Random Variable (Noise)

Decoder

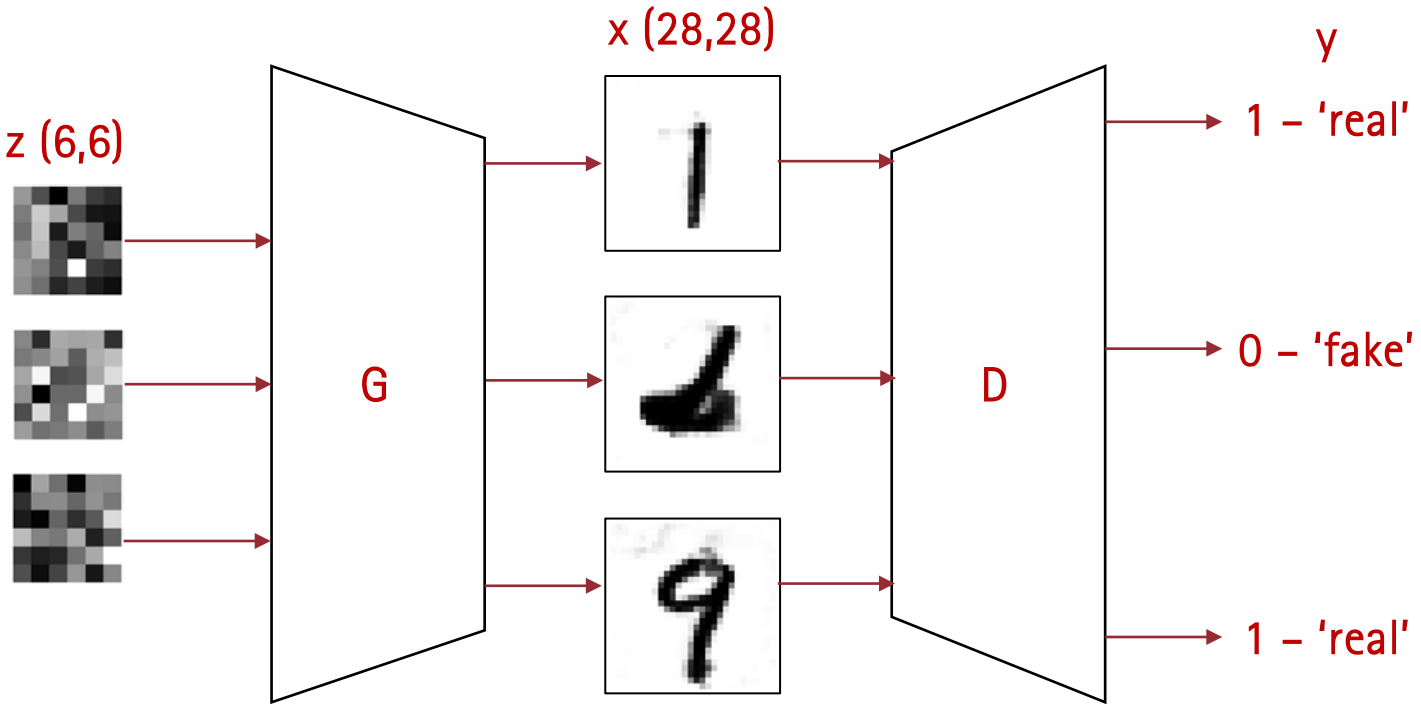




Both networks (generator and discriminator) are trained in alternating steps until the generator produces good quality images.

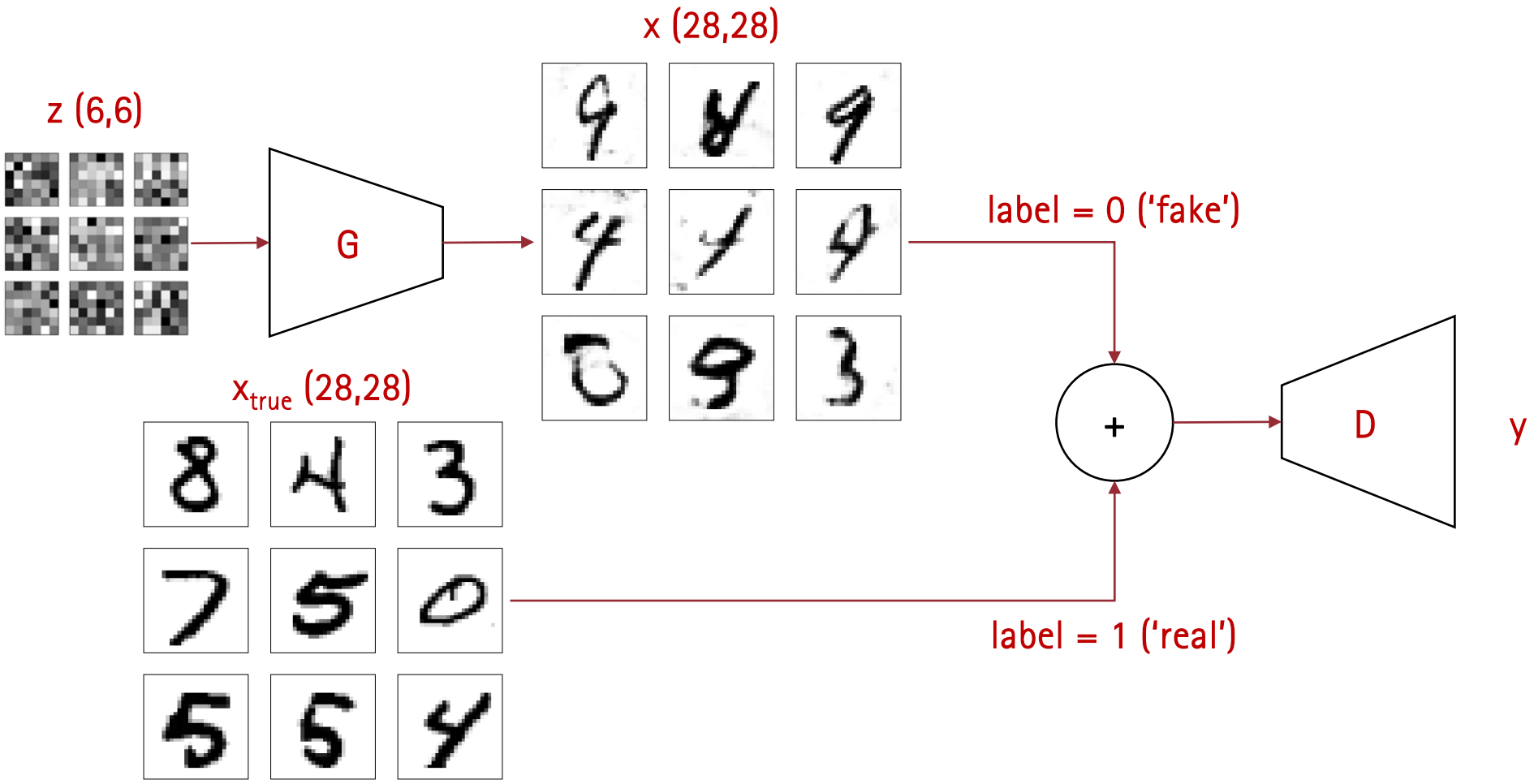
Train strategy

- GANs are based on a **game scenario** in which the **generator network** must **compete** against an adversary
- The **generator creates** samples **(x)** from an input **noise (z)** and is **trained to deceive** the **discriminator** by producing images that are **difficult to distinguish** from **real images**
  - **Generator block:** $x = G(z)$
  - **Discriminator block:** $y = D(x)$



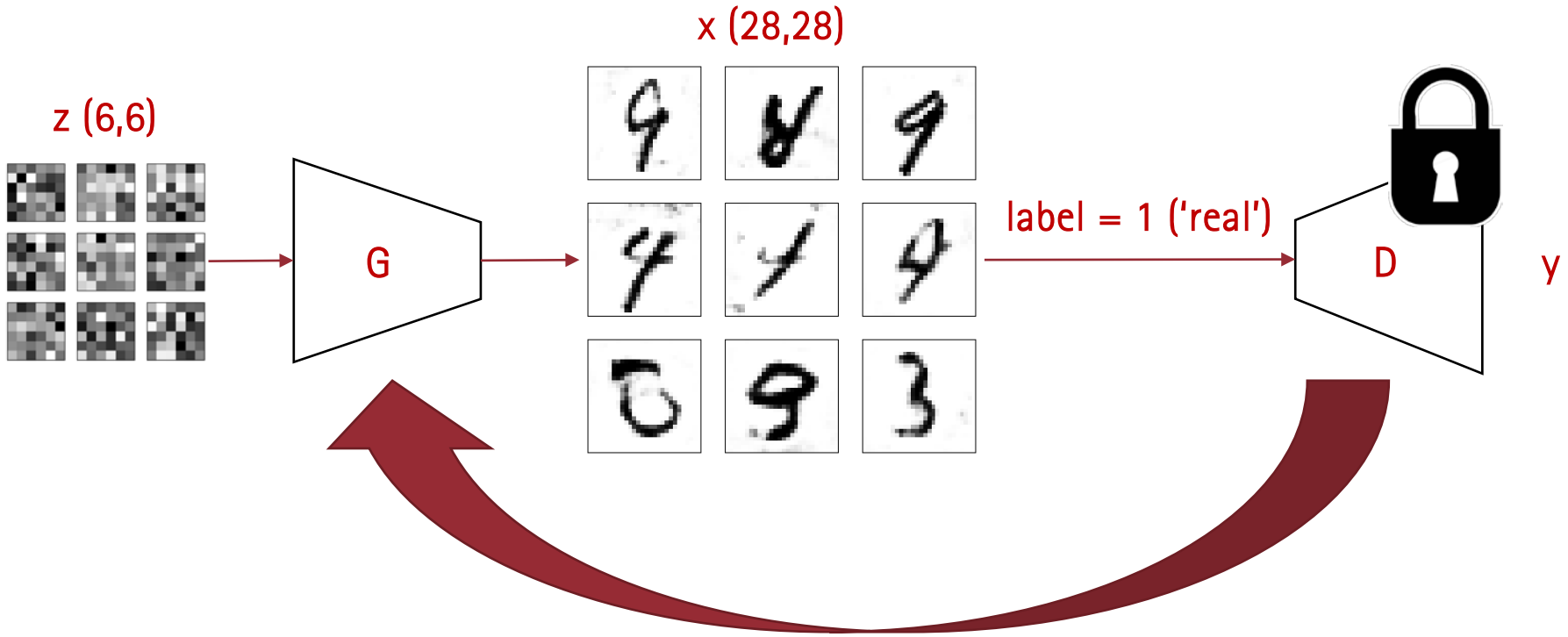
Train strategy

- PHASE 1: For each **batch**, **train** of the **discriminator** by declaring the **generated images as fake**, and the **images from the dataset as real**



Train strategy

- PHASE 2: **Freezing** the weights of the **discriminator**, train the network **declaring the labels** of the generated images as **real**. The **gradient** will update the **generator weights** in order to **minimize the error** between the **predicted label** and the **"true label"** (forced to real).



Train procedure (for each batch)

- Divide the GAN into the generator and the discriminator

```
generator, discriminator = gan.layers
```

- Generate fake images, mix the fake images with real ones and define the labels as '0' for fake and '1' for real.

```
fake_images = generator(noise)
mixed_images = tf.concat([fake_images, real_images], axis=0)
discriminator_labels = tf.constant([[0.]] * batch_size + [[1.]] * batch_size)
```

- Train the discriminator with the set created.

```
discriminator.trainable = True
discriminator.train_on_batch(mixed_images, discriminator_labels)
```

- Freeze the discriminator weights.

```
discriminator.trainable = False
```

- Generate fake images with the generator, and define the labels as '1'.

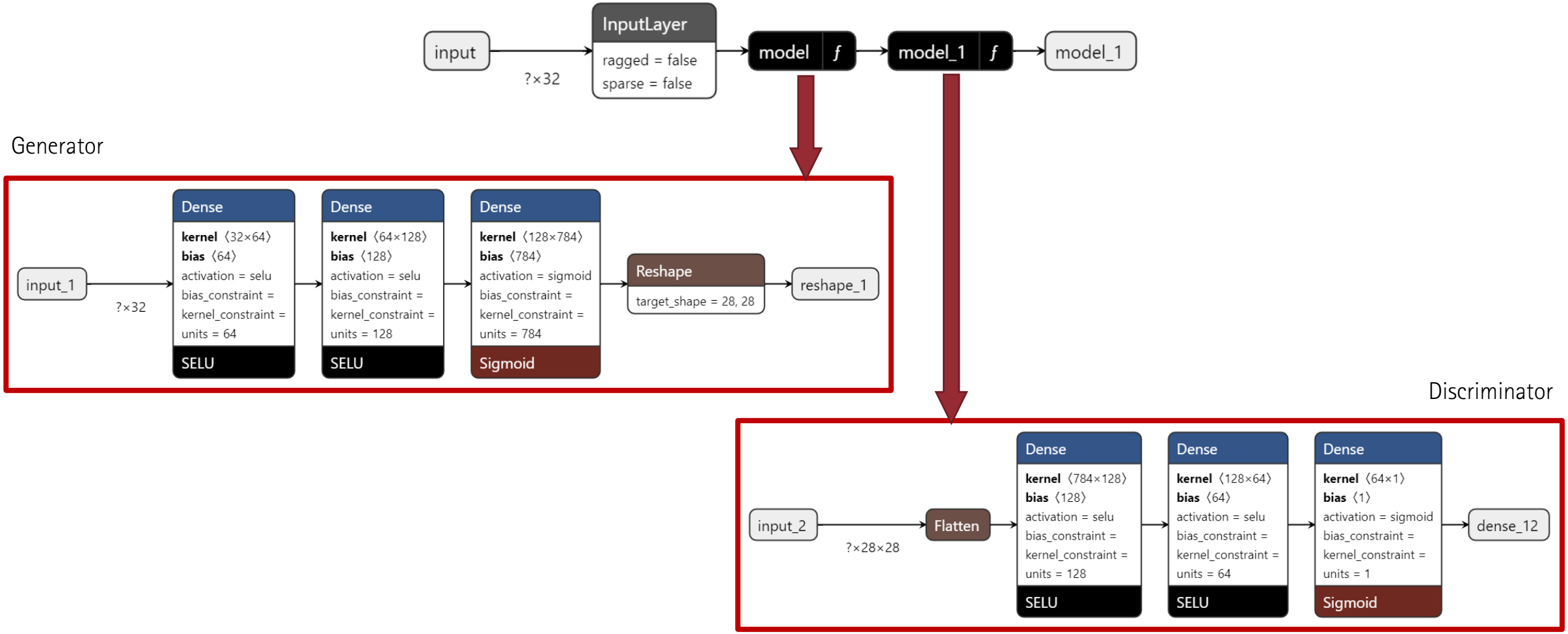
```
noise = tf.random.normal(shape=[batch_size, random_normal_dimensions])
generator_labels = tf.constant([[1.]] * batch_size)
```

- Train the complete network with the set created.

```
gan.train_on_batch(noise, generator_labels)
```

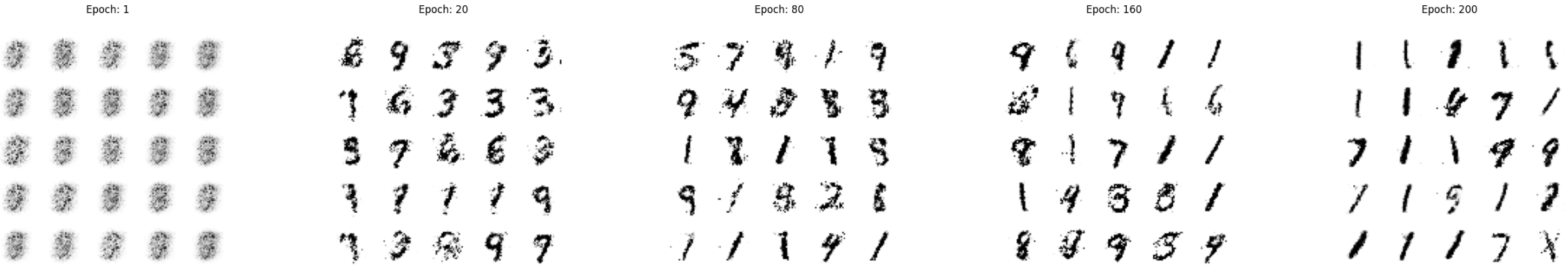
2.2 Dense GAN for image generation

- Model of a Dense GAN.

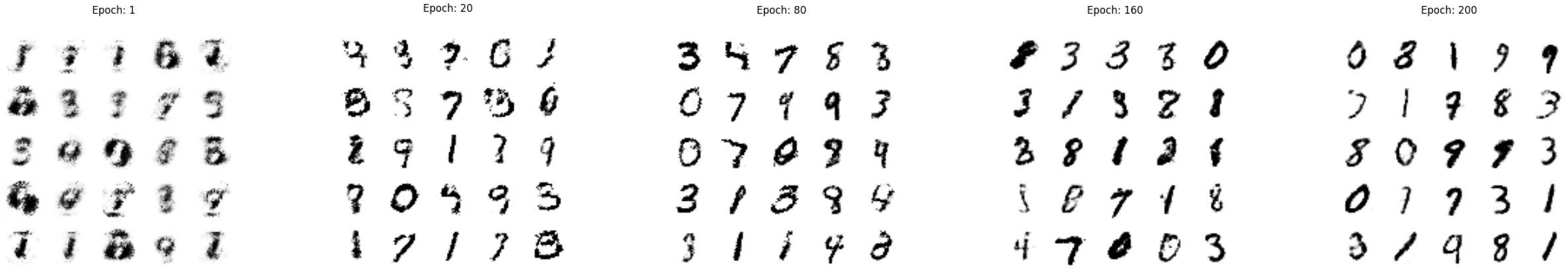


2.2 Dense GAN for image generation

- 64 → 128 →128 → 64

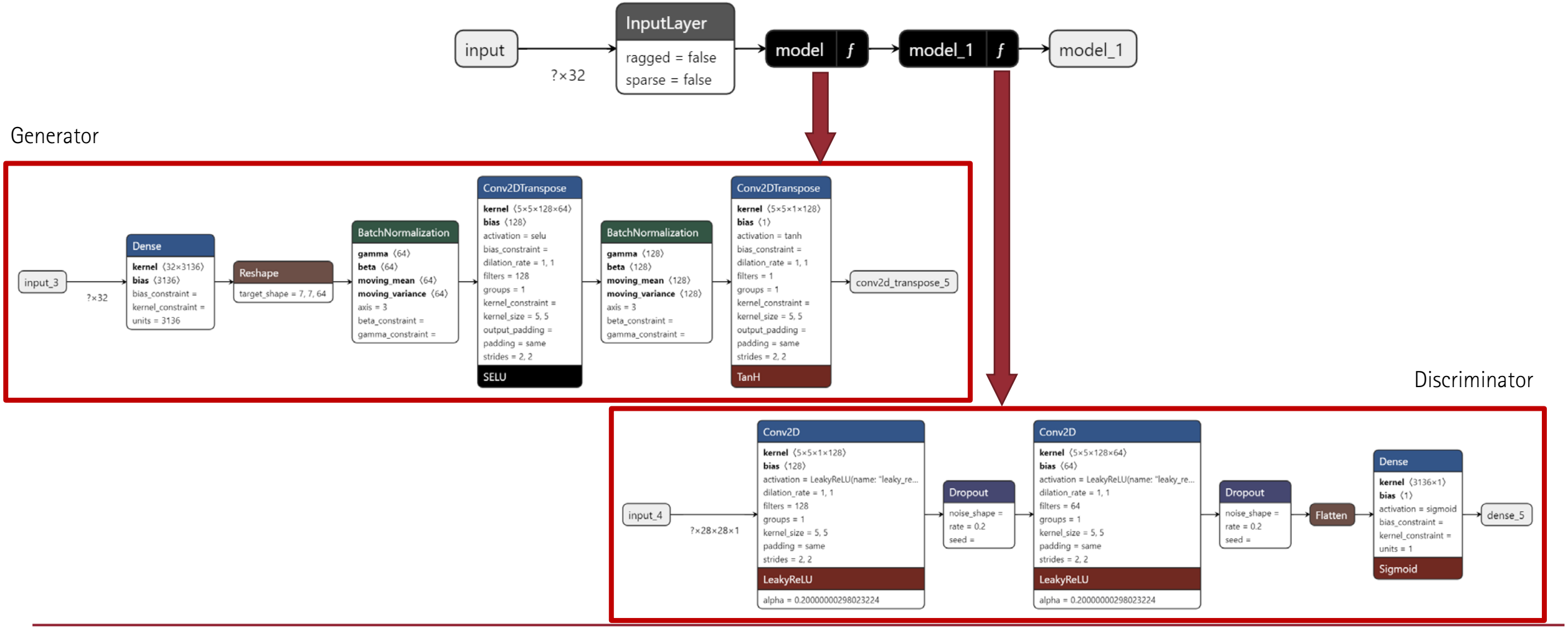


- 256 → 512 → 512 → 256



2.3 Convolutional GAN for image generation

- Model of a Convolutional GAN.

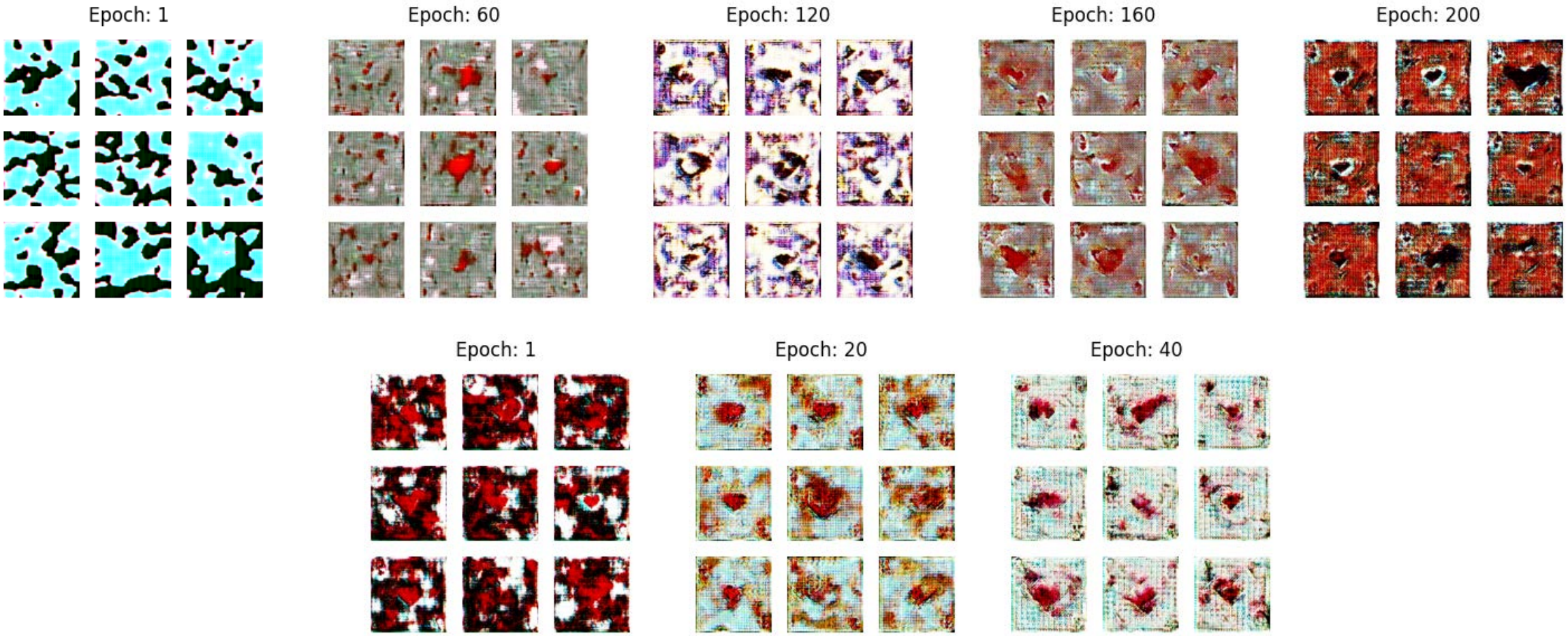


2.3 Convolutional GAN for image generation

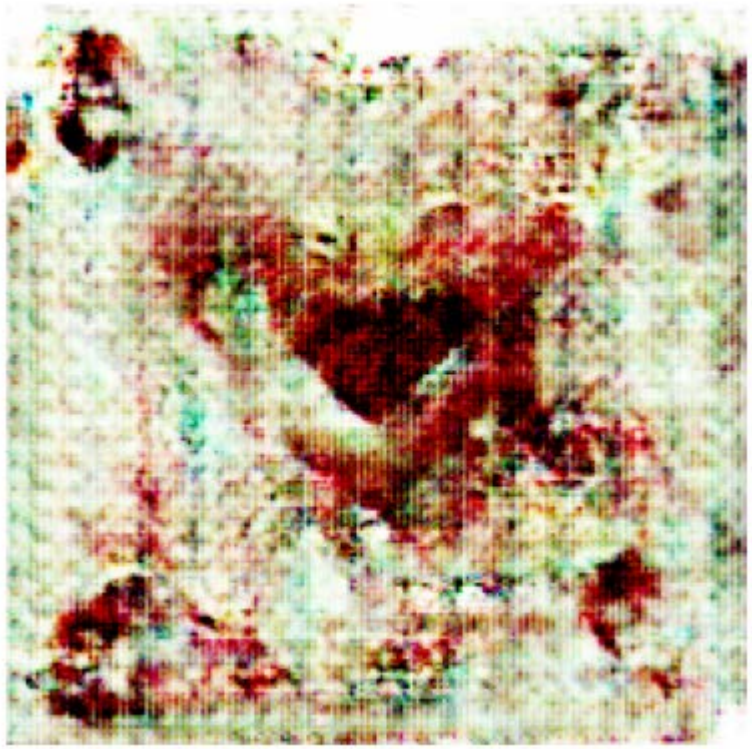
- 64 → 128 →128 → 64

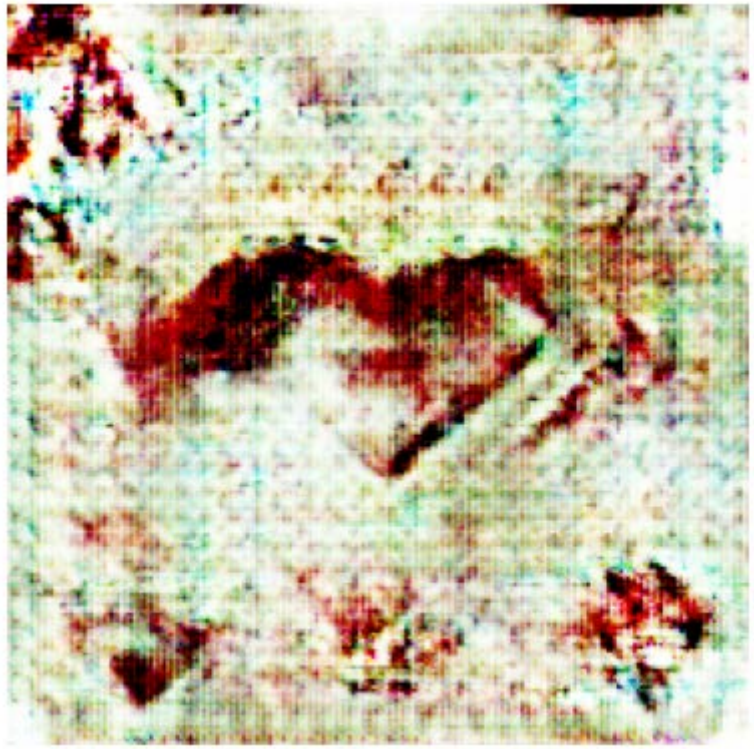
- 256 → 512 → 512 → 256

Example: Cards generation with original ConvNet



Example: Cards generation with original ConvNet





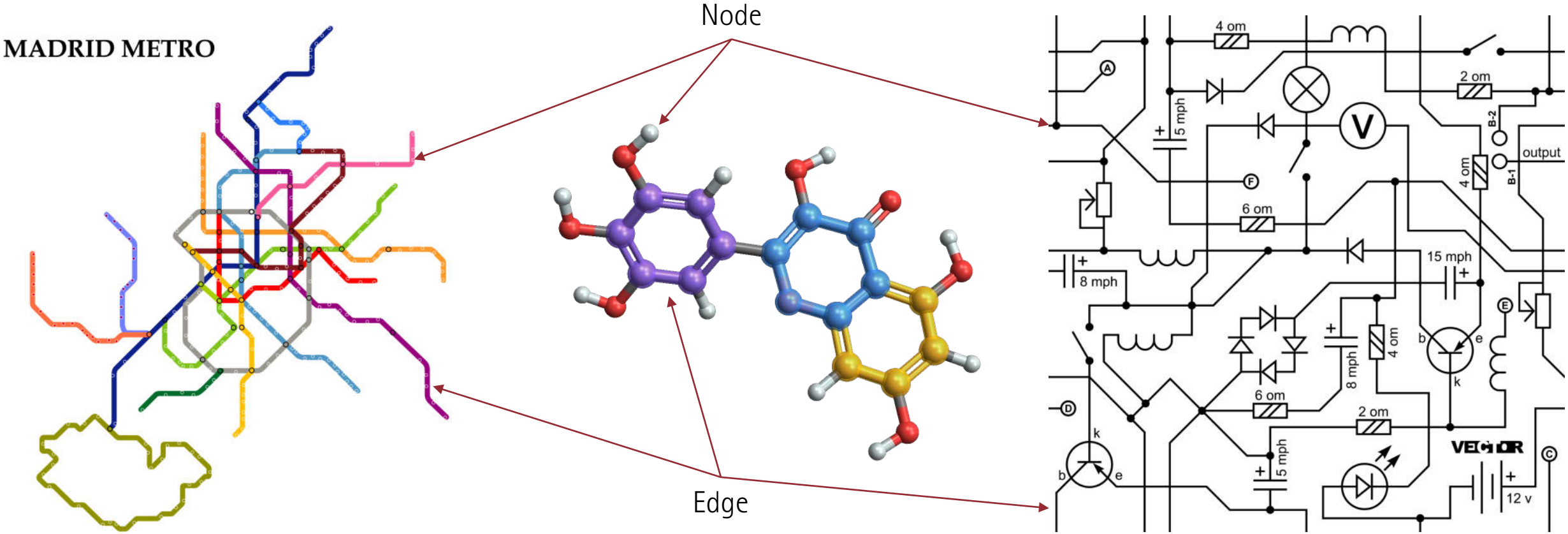
1. In the case of the MNIST dataset, 28x28 pixel images are generated, resulting in 784 pixels
2. In the case of the cards, 224x224 pixel images are generated, which is 50,176 pixels (64 times more information) and we are doing this with only 3 layers, resulting in a shallow network with few parameters
3. We generate grayscale images in MNIST and RGB images in the cards

3. GNN – Graph Neural Networks

-PAPER: Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., & Monfardini, G. (2008). The graph neural network model. IEEE transactions on neural networks, 20(1), 61-80.

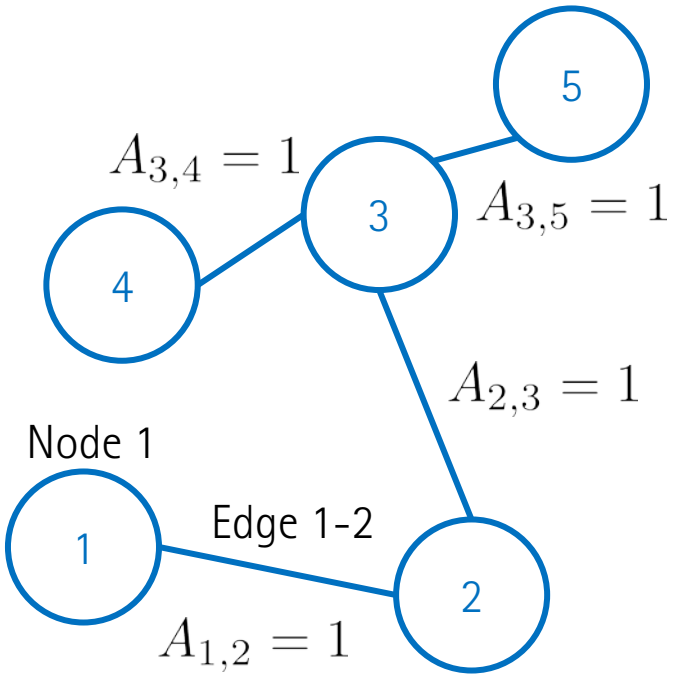
Otras bibliografías:
-Prince, S. J. (2023). Understanding deep learning. MIT press.
-Hamilton, W. L. (2020). Graph representation learning. Morgan & Claypool Publishers.
-Baldi, P. (2021). Deep learning in science. Cambridge University Press.

Graphs information: Nodes



- Message passing: theoretical mechanism by which nodes can communicate with each other in a graph
- GNN: deep learning model that automates and learns the best way to do things for a particular task
  - GNNs are designed to learn complex patterns in graph data that would be difficult or impossible to program manually

Adjacency Matrix



$$A_{i,j}$$

$$\begin{array}{c|ccccc} & \text{N1} & \text{N2} & \text{N3} & \text{N4} & \text{N5} \\ \hline \text{Node 1} & 0 & 1 & 0 & 0 & 0 \\ \text{Node 2} & 1 & 0 & 1 & 0 & 0 \\ \text{Node 3} & 0 & 1 & 0 & 1 & 1 \\ \text{Node 4} & 0 & 0 & 1 & 0 & 0 \\ \text{Node 5} & 0 & 0 & 1 & 0 & 0 \end{array}$$

$$A_{i,j} = 0 \quad \forall \quad i = j$$
$$A_{i,j} = 1 \quad \text{if } i \text{ is connected to } j$$
$$A_{i,j} = 0 \quad \text{otherwise}$$
$$A_{i,j} = A_{j,i} \quad \text{not oriented edges}$$

Simple message passing

$$\underset{A}{\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}} \underset{H_0}{\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}} = \underset{H_1}{\begin{pmatrix} 2 \\ 4 \\ 11 \\ 3 \\ 3 \end{pmatrix}}$$

Node features

$$D_{i,j}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

$$D_{i,j} = 0 \quad \forall \quad i \neq j$$
$$D_{i,i} = n \quad \text{n: number of edges of node n}$$

Dimension matrix
(to normalize node effects)

Message passing with
dimension matrix

$$\underbrace{\underset{D^{-1}}{\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}} \underset{A}{\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}}}_{A_{\text{avg}}} \underset{H_0}{\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}} = \underset{H_1}{\begin{pmatrix} 2 \\ 2 \\ 3.67 \\ 3 \\ 3 \end{pmatrix}}$$

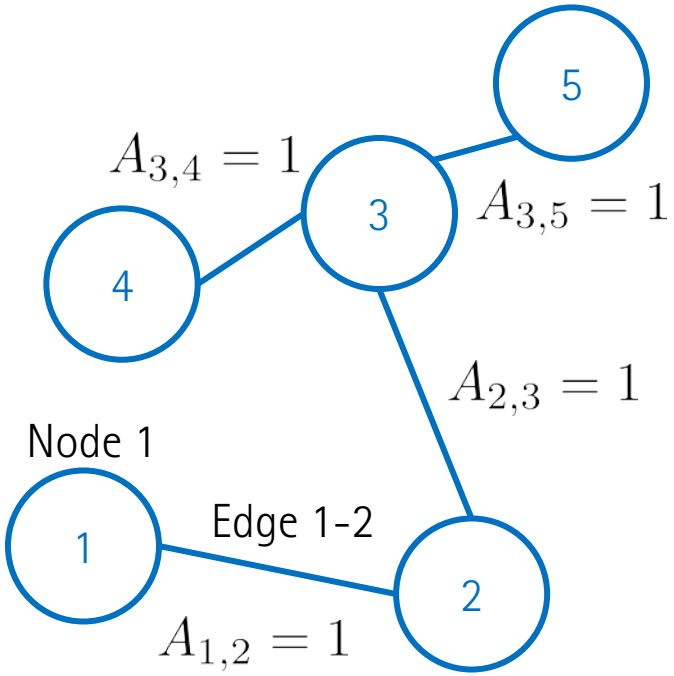
$$A_{\text{avg}}$$

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1/2 & 0 & 1/2 & 0 & 0 \\ 0 & 1/3 & 0 & 1/3 & 1/3 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Normalized Adjacency Matrix

Feature of the node itself, its own connection

$$\tilde{A} = A + I = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix} \qquad \tilde{D} = D + I = \begin{pmatrix} 2 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 4 & 0 & 0 \\ 0 & 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 & 2 \end{pmatrix}$$



Helps maintain the variance/scale of node features as it propagates

Normalize both the rows and columns of the adjacency matrix

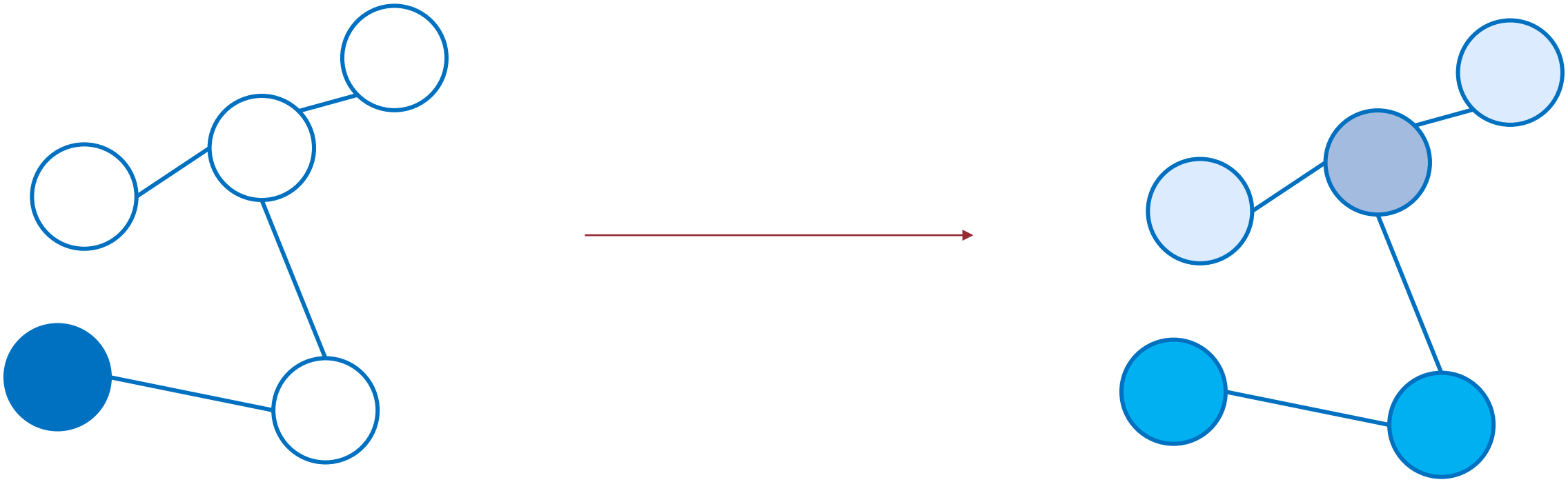
$$\hat{A} = \tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}}$$

$$\begin{pmatrix} 0.5 & 0.41 & 0 & 0 \\ 0.41 & 0.33 & 0.29 & 0 \\ 0 & 0.29 & 0.25 & 0.35 \\ 0 & 0 & 0.35 & 0 \\ 0 & 0 & 0.35 & 0.5 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}} \end{pmatrix}$$

Message passing with Normalized Adjacency Matrix

$$\underbrace{\begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}} \end{pmatrix}}_{\tilde{D}^{-\frac{1}{2}}} \underbrace{\begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \end{pmatrix}}_{\tilde{A}} \underbrace{\begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{\sqrt{3}} & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{\sqrt{2}} \end{pmatrix}}_{\tilde{D}^{-\frac{1}{2}}} \underbrace{\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{pmatrix}}_{H_0} = \underbrace{\begin{pmatrix} 1.32 \\ 1.94 \\ 4.51 \\ 3.06 \\ 3.56 \end{pmatrix}}_{H_1}$$

Water drop example



Water drop example code

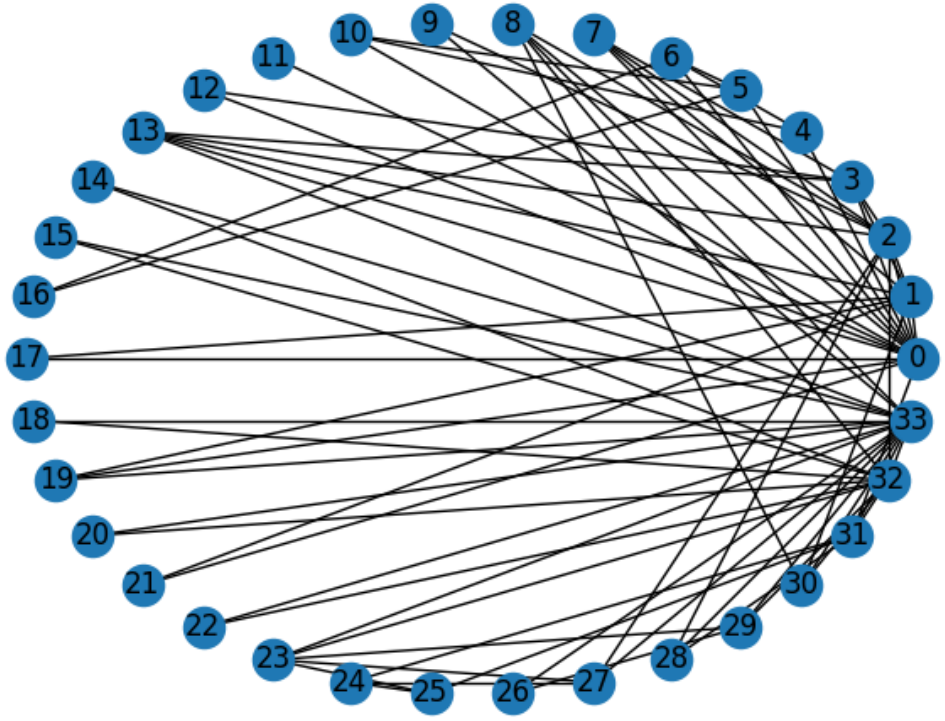
```
A = np.array([[0, 1, 0, 0, 0],[1, 0, 1, 0, 0],[0, 1, 0, 1, 1],[0, 0, 1, 0, 0],
              [0, 0, 1, 0, 0]])          # adjacency matrix

A_mod = A + np.eye(g.number_of_nodes()) # adding self-loops

D_mod = np.zeros_like(A_mod)            # dimension matrix with self-loops
np.fill_diagonal(D_mod, A_mod.sum(axis=1).flatten())

# A_hat calculation
D_mod_invroot = np.linalg.inv(sqrtm(D_mod))
A_hat = D_mod_invroot @ A_mod @ D_mod_invroot

H = np.zeros((g.number_of_nodes(), 1))  # node features, starting all in 0
H[0,0] = 1                              # the "water drop" in node 1
iters = 10                              # iterations
results = [H.flatten()]                 # first state as a list
for i in range(iters):
    H = A_hat @ H                       # new features: A_hat @ H
    results.append(H.flatten())         # adding this features to the list
```

GCNN from scratch with Numpy: Dataset

- Paper: Zachary W. (1977). An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33, 452-473.

- Classic study in the field of anthropology and social network análisis
- Observed social interactions within a karate club over a three-year period and recorded friendships and conflicts among the club's 34 members
- The study focuses on how interactions and information flows can lead to conflicts and eventually to the splitting (or fission) of a social group

- Dataset: Zachary's Karate Club: https://networkx.org/documentation/stable/auto_examples/graph/plot_karate_club.html
  - 34 nodes
  - 78 edges representing friendships
  - Not directed graph, if A is friend of B, then B is friend of A.



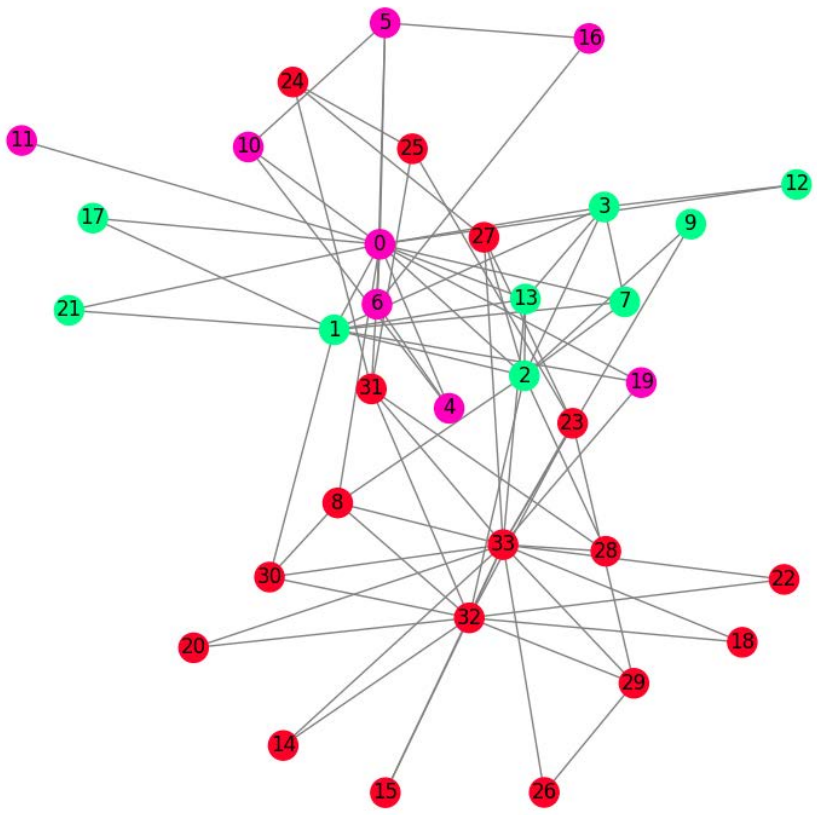
A matrix =

```
array([[0., 4., 5., ..., 2., 0., 0.],
       [4., 0., 6., ..., 0., 0., 0.],
       [5., 6., 0., ..., 0., 2., 0.],
       ...,
       [2., 0., 0., ..., 0., 4., 4.],
       [0., 0., 2., ..., 4., 0., 5.],
       [0., 0., 0., ..., 4., 5., 0.]])
```

GCNN from scratch with Numpy

- Input: from previous graph, normalized adjacency matrix *A* and its dependencies
- Goal: to classify nodes

- Nodes are defined into 3 classes (colors) using the Greedy Modularity Communities algorithm
    - Algorithm used to detect communities within a graph, based on the optimization of the modularity of the network
    - A high modularity value indicates a strong community structure
    - A "strong community structure" refers to nodes within the same community tend to be more densely connected to each other compared to nodes from different communities

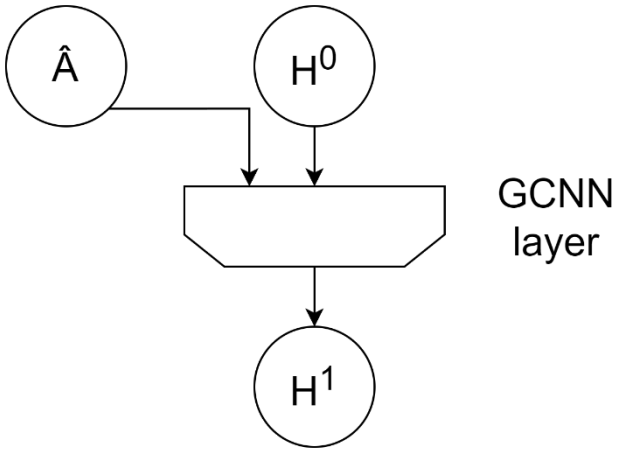


- This is going to be our ground truth labels

- Goal of the training: giving the features matrix and the adjacency matrix (message passing), the GCN trains the weights to predict the GT
- Inference: another graph (other *A* matrix) can be used and the GCN would reach the optimal structure

GCNN from scratch with Numpy: GCNN Layer

- Implementation of the message passing algorithm through the layer:



$$H^1 = \sigma\left(W\hat{A}H^0\right)$$

Activation fn

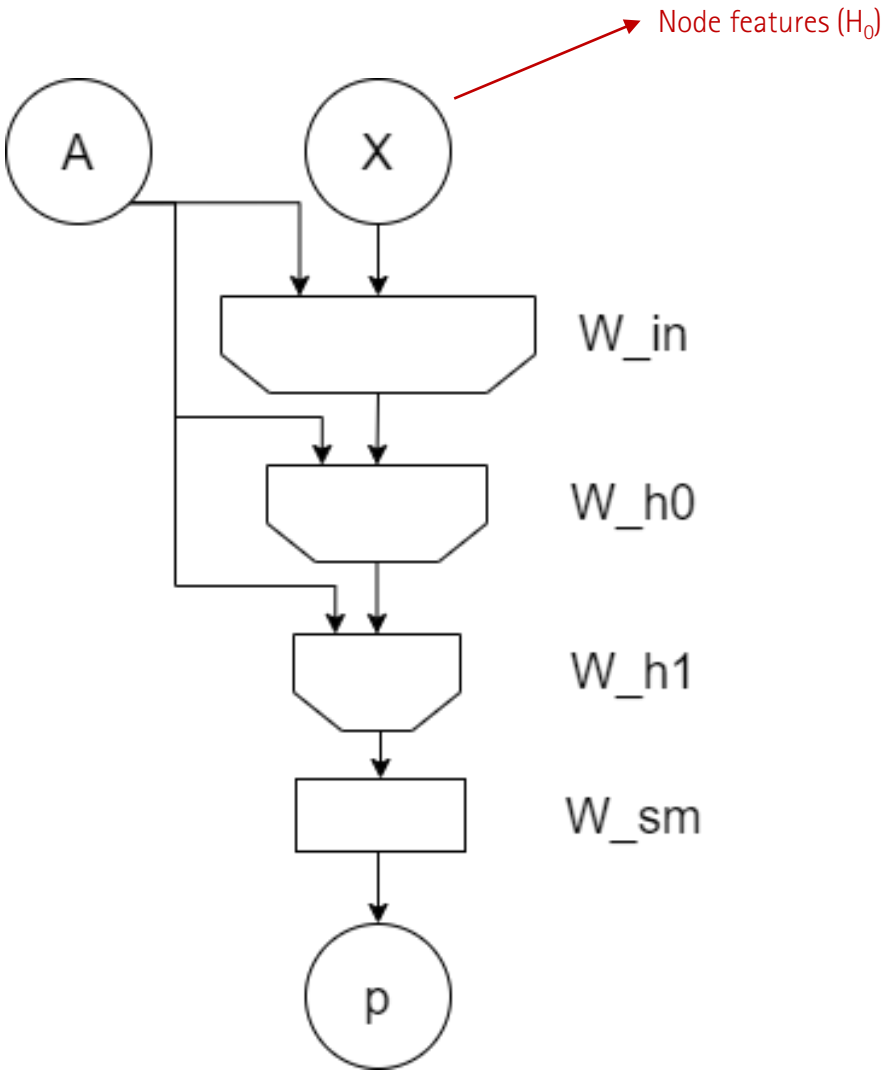
Layer weights

Message passing

```
def forward(self, A, X, W, activation):
        # A: normalized adjacency
        # X: input to layer (H0)
        # W: weights of the layer
        # activation
         H = W @ (A @ X)
         H = activation(H)
         return H
```

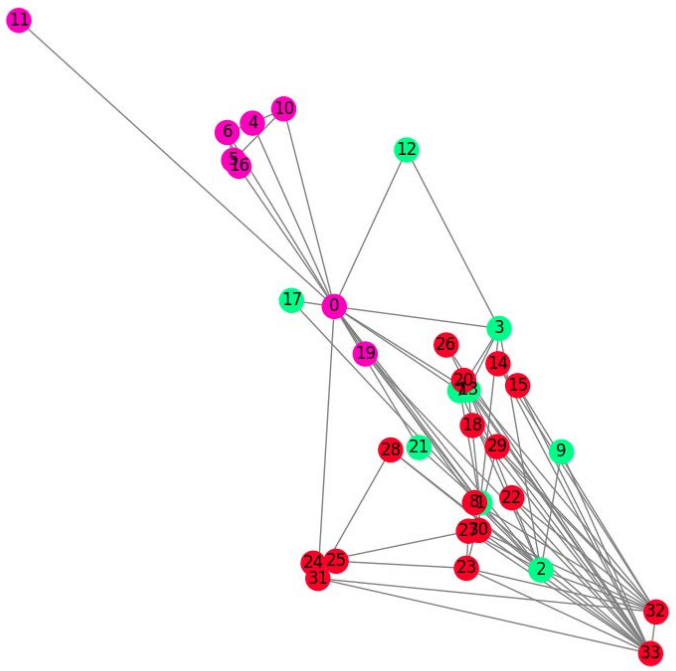
GCNN from scratch with Numpy: Model

- Model:
  - GCNN Input layer (W_in), n_outputs = 8:
    - Input: X (34x34), A (34x34)
    - Output: H_in (34x8)
  - GCNN Hidden layer (W_h0), n_outputs = 4
    - Input: H_in (34x8), A (34x34)
    - Output: H_0 (34x4)
  - GCNN Hidden layer (W_h1), n_outputs = 2 (to draw in 2D)
    - Input: H_0 (34x4), A (34x34)
    - Output: H_1 (34x2)
  - MLP Output layer (W_sm), n_outputs = 3 (one for each class)
    - Input: H_1 (34x2)
    - Output: H_sm (34x3)
    - Simple MLP layer with softmax activation

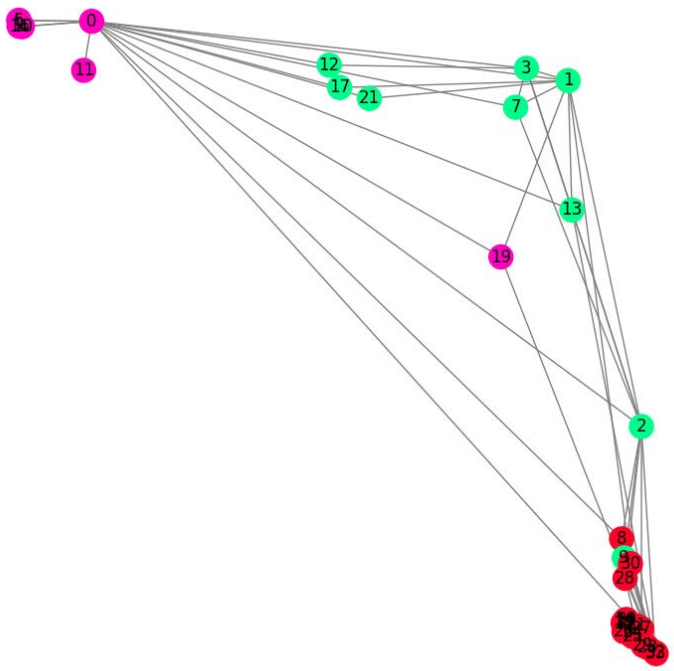


GCNN from scratch with Numpy: Result

- GCNN output without training



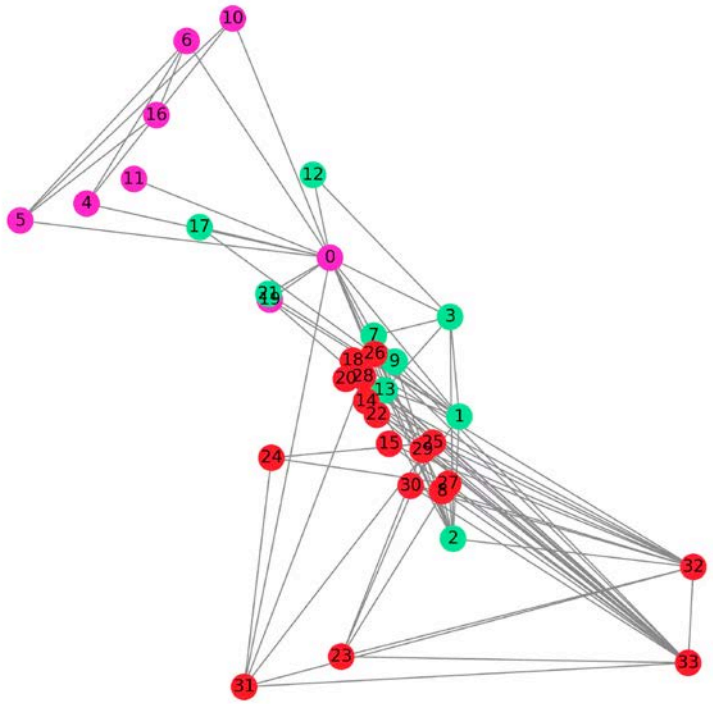
- Trained GCNN



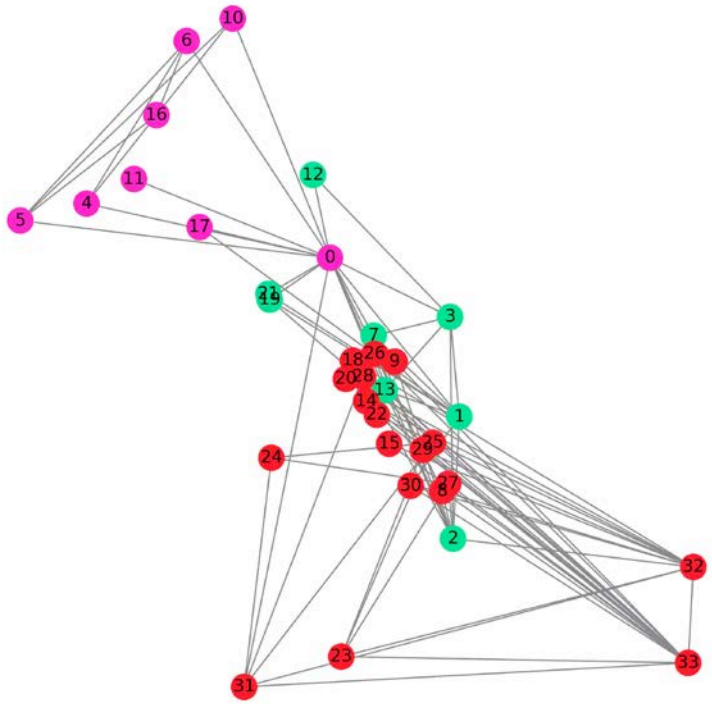
The draw of the positions is done by taking the 2D embeddings output from h1 layer.
The network "learns" to position the nodes close to their relatives in the graph

GCNN from scratch with Numpy: Learning process video

2D positions predictions with color GT labels



2D positions predictions with color predicted labels

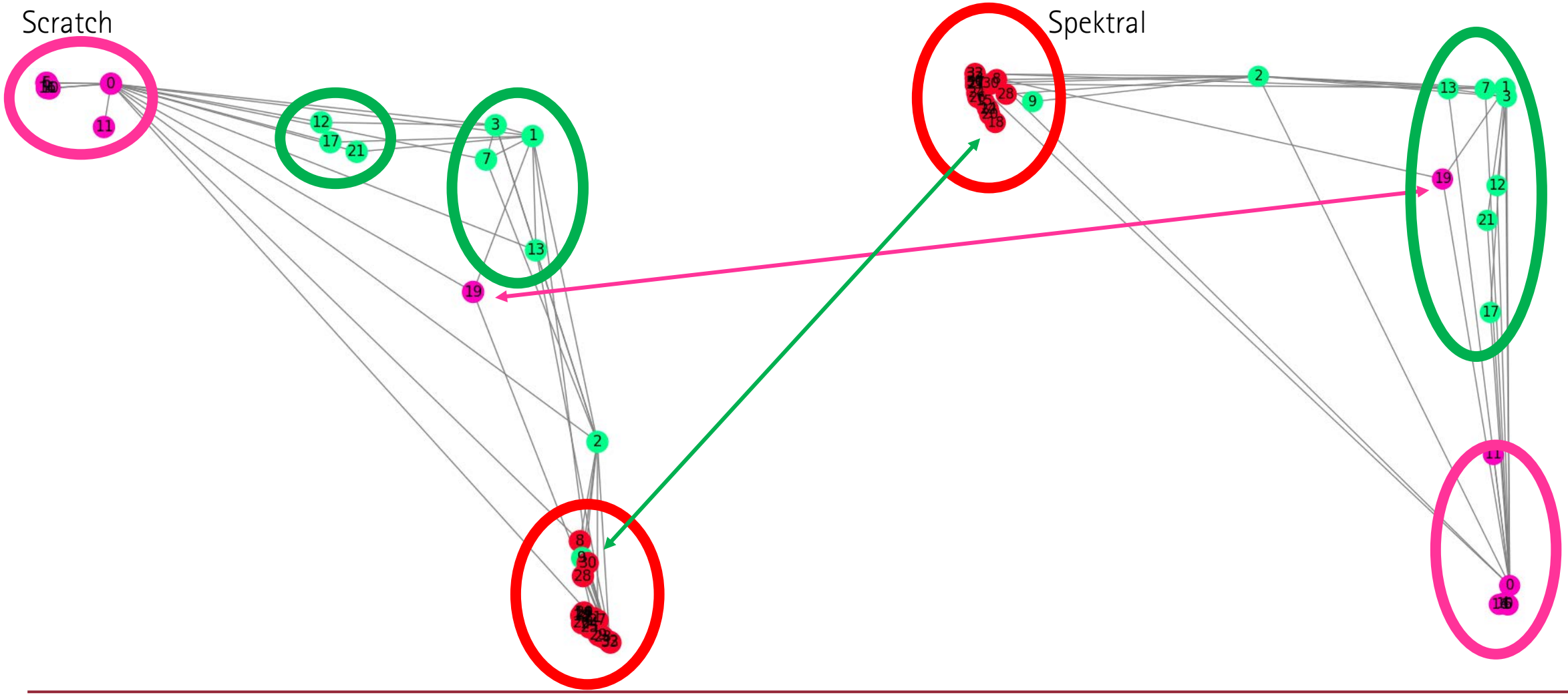


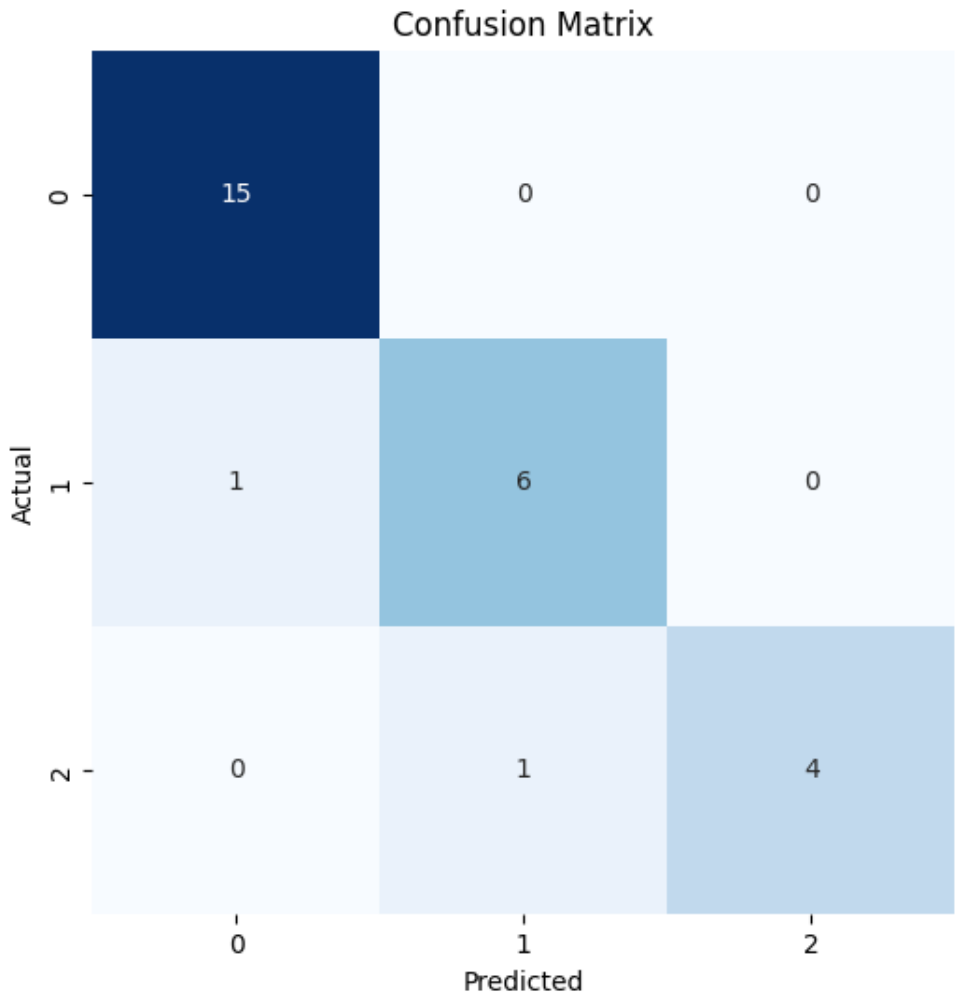
GCNN with Spektral library[1]:

- Spektral is a Python library for Graph Deep Learning, based on Keras and Tensorflow 2. Installable with “pip install spektral”
- Includes a keras-compatible layer for Graph Convolutional Networks: “from spektral.layers import GCNConv”
- Straightforward model definition using the Keras Functional API
- Standard optimizers and losses from Tensorflow
- Standard training with fit method
- Model definition and training is reduced to less than 15 lines

[1]https://graphneural.network/

Scratch vs. Spektral results



GCNN metrics

	precision	recall	f1-score	support
0.0	0.94	1.00	0.97	15
1.0	0.86	0.86	0.86	7
2.0	1.00	0.80	0.89	5
accuracy			0.93	27
macro avg	0.93	0.89	0.90	27
weighted avg	0.93	0.93	0.92	27